Neural Networks

Ways to group machine learning models

How do you acquire training data?

Supervised
Unsupervised
Reinforcement

What are you outputting?

Regression
Classification/Clustering

How are you modeling? New!

Discriminative
Generative

Generative Models

- Naïve Bayes, Bayes classifiers are generative models
- Learn the model for each class y given input features x p(x|y). (The likelihood probability)

- To do classification we want to solve for the best y given input feature x (the posterior)

$$y^* = argmax_y P(y|x)$$

- We can use Bayes' rule

$$y^* = argmax_y \frac{P(x|y)P(y)}{P(x)}$$

Generative Models

$$y^* = argmax_y \frac{P(x|y)P(y)}{P(x)}$$

- P(y) is called the prior probability
- P(x) is ignored since we only care for argmax wrt. Y

- Can we use P(y|x) instead?

$$y^* = argmax_y P(y|x)$$

Discriminative models

- Discriminative models model P(y|x) directly

$$y^* = argmax_y P(y|x)$$

- P(y|x) is called the <u>posterior probability</u>
- Generally, P(y|x) can be any function h(x,y) that gives a score for each class
  - Logistic regression
  - SVM
  - Neural networks

Discriminative vs Generative

- Model the posterior P(y|x)
- Care about how to *discriminate* between different classes
- Usually outperforms generative models in classification tasks
- Need to retrain the whole model

- Model the likelihood P(x|y) Learns about how x is *generated* from y
- Worse classification performance. Mismatch between training objective
- Easy to add a new class y' – train P(x|y = y') Example: Adding a new class in Naïve Bayes
- Can generate x by sampling from P(x|y)

Notes on Generative modeling

- Some people say generative models model the joint probability, $p(x,y)$.
- This is also true because when we model $p(x|y)$, we also model the prior $p(y)$.
  - $p(x,y) = p(x|y)p(y)$

- With this view,
  - Generative models use the joint probability $p(x,y)$
  - Discriminative models use the conditional probability $p(y|x)$

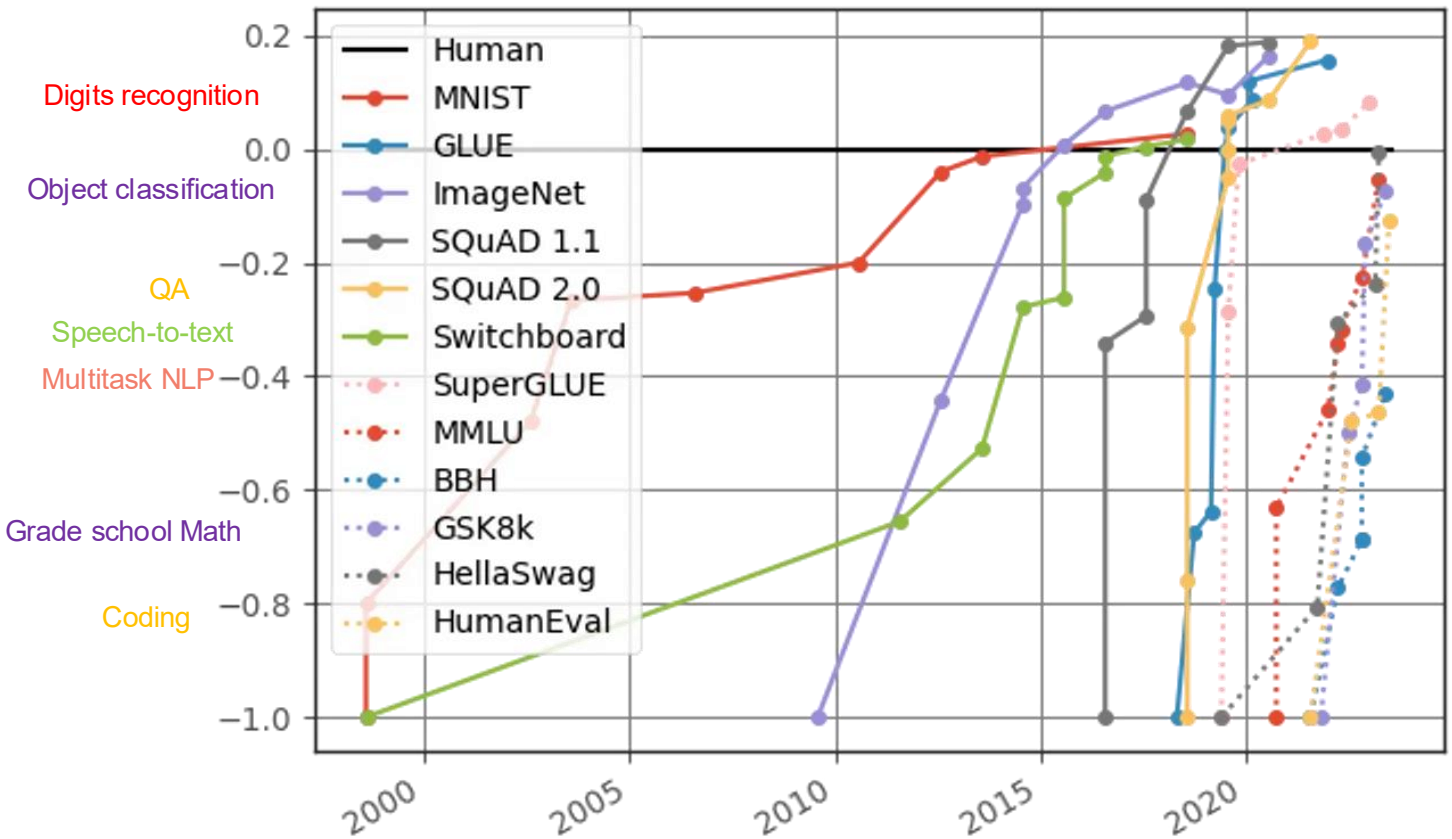
Generating data from generative model?

- We have the joint p(x,y) so we can sample from the distribution for a new (x,y) pair.
- This generates a new data sample x
  - You cannot sample from the posterior p(y|x) because you do not know p(x).

NEURAL NETWORKS

Deep learning = Deep neural networks = neural networks

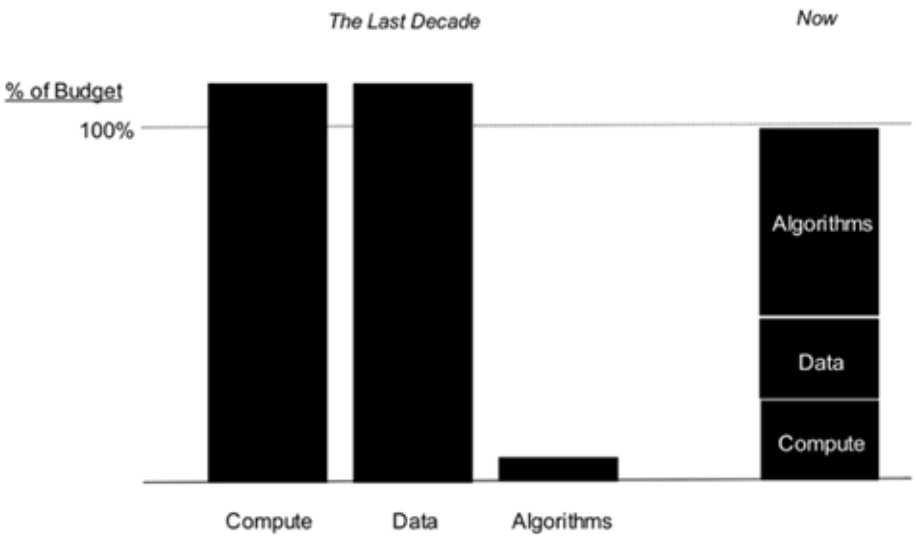
Neural networks saturating benchmarks



https://contextual.ai/blog/plotting-progress-in-ai

Why now

- Neural Networks has been around since 1990s
- Big data – DNN can take advantage of large amounts of data better than other models
- GPU – Enable training bigger models possible
- Deep – Easier to avoid bad local minima when the model is large



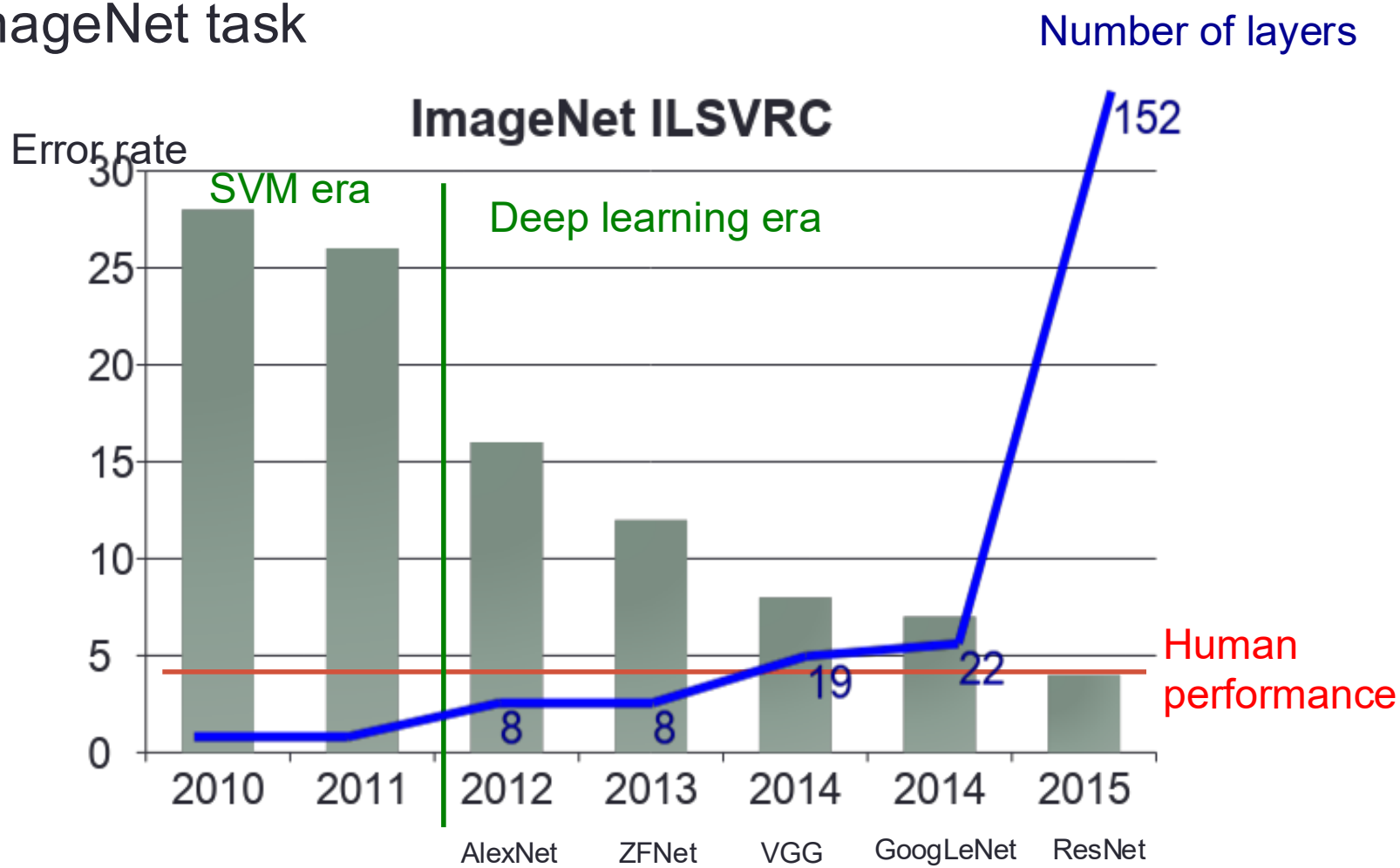
http://www.kdnuggets.com/2017/06/practical-guide-machine-learning-understand-differentiate-apply.html

ImageNet - Object classification

Alex, Krizhevsky, Imagenet classification with deep convolutional neural networks, 2012

Wider and deeper networks

- ImageNet task



Olga Russakovsky, ImageNet Large Scale Visual Recognition Challenge, 2014 https://arxiv.org/abs/1409.0575

arXiv.org > stat > arXiv:1806.05393

Search or Article ID | All fields

(Help | Advanced search)

Statistics > Machine Learning

Dynamical Isometry and a Mean Field Theory of CNNs: How to Train 10,000-Layer Vanilla Convolutional Neural Networks

Lechao Xiao, Yasaman Bahri, Jascha Sohl-Dickstein, Samuel S. Schoenholz, Jeffrey Pennington

(Submitted on 14 Jun 2018)

In recent years, state-of-the-art methods in computer vision have utilized increasingly deep convolutional neural network architectures (CNNs), with some of the most successful models employing hundreds or even thousands of layers. A variety of pathologies such as vanishing/exploding gradients make training such deep networks challenging. While residual connections and batch normalization do enable training at these depths, it has remained unclear whether such specialized architecture designs are truly necessary to train deep CNNs. In this work, we demonstrate that it is possible to train vanilla CNNs with ten thousand layers or more simply by using an appropriate initialization scheme. We derive this initialization scheme theoretically by developing a mean field theory for signal propagation and by characterizing the conditions for dynamical isometry, the equilibration of singular values of the input-output Jacobian matrix. These conditions require that the convolution operator be an orthogonal transformation in the sense that it is norm-preserving. We present an algorithm for generating such random initial orthogonal convolution kernels and demonstrate empirically that they enable efficient training of extremely deep architectures.

Comments: ICML 2018 Conference Proceedings
Subjects: **Machine Learning (stat.ML)**; Machine Learning (cs.LG)
Cite as: arXiv:1806.05393 [stat.ML]
(or arXiv:1806.05393v1 [stat.ML] for this version)

Submission history

From: Samuel Schoenholz [view email]
[v1] Thu, 14 Jun 2018 07:04:15 GMT (6734kb,D)

Which authors of this paper are endorsers? | Disable MathJax (What is MathJax?)

Download:
- PDF
- Other formats

(license)

Current browse context:
stat.ML
< prev | next >
new | recent | 1806

Change to browse by:
cs
cs.LG
stat

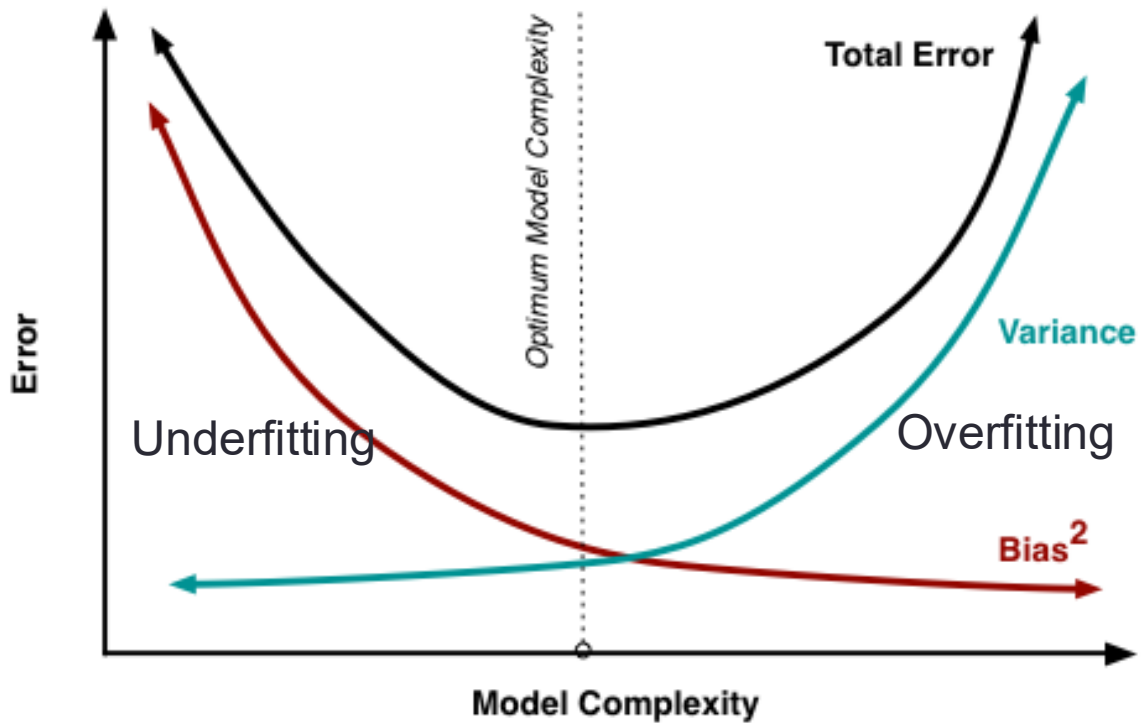
References & Citations
- NASA ADS

Bookmark (what is this?)

Link back to: arXiv, form interface, contact.

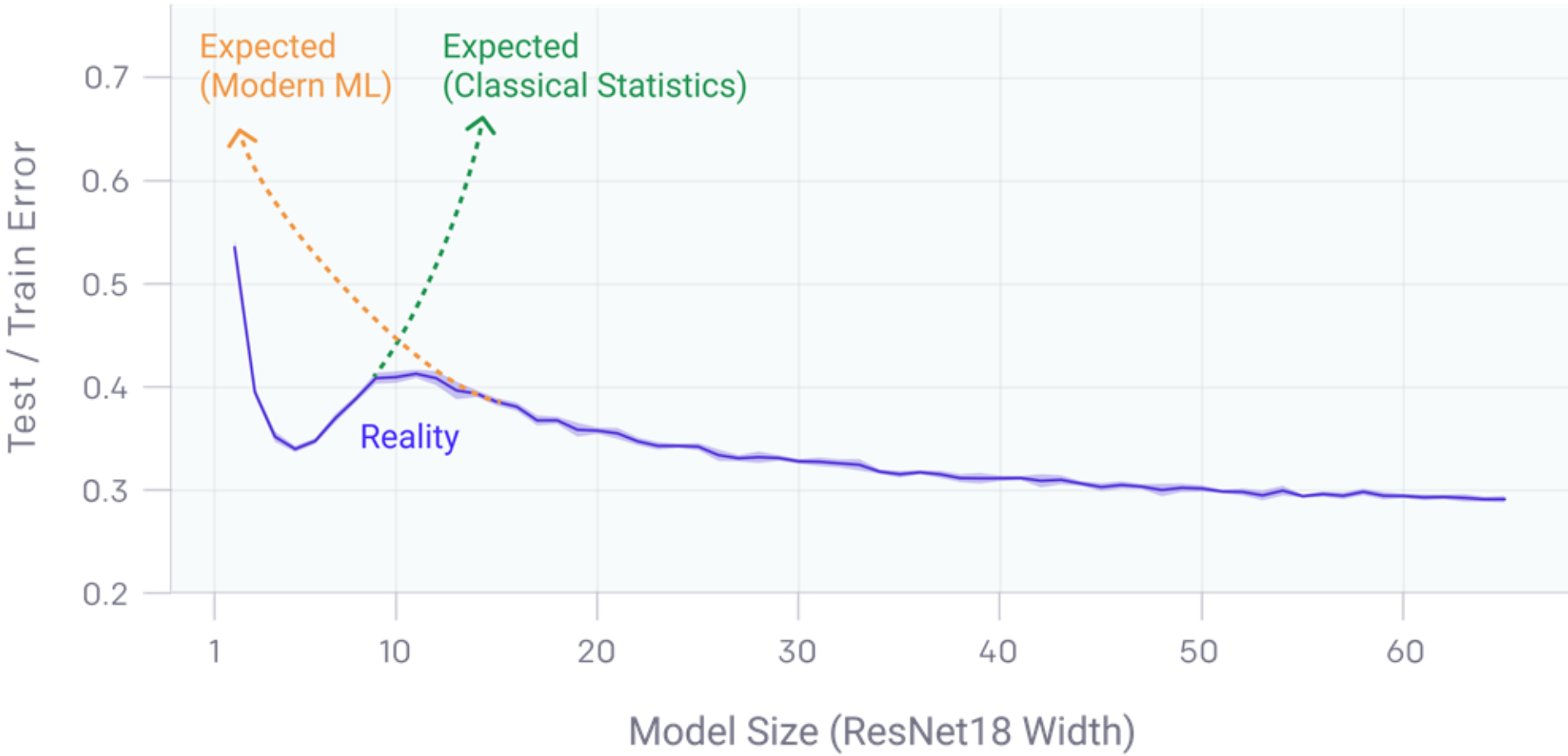
Bias-Variance Underfitting-Overfitting

- Usually if you try to reduce the bias of your model, the variance will increase, and vice versa.
- Called the bias-variance trade-off



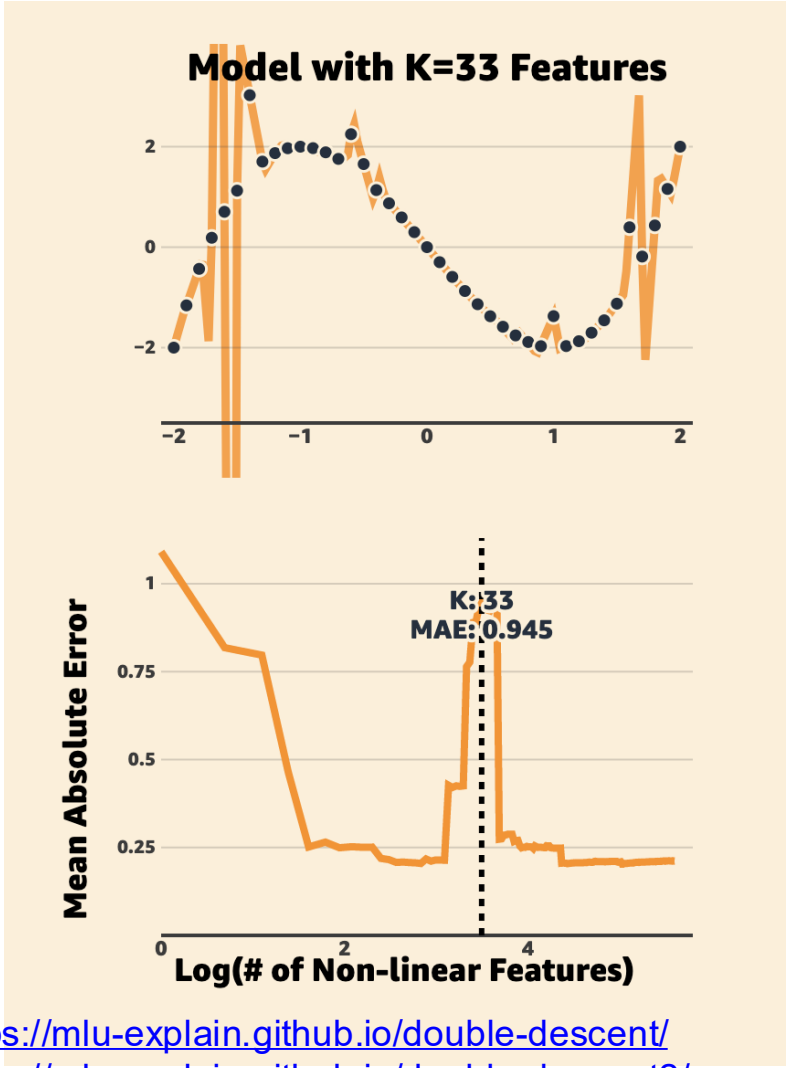
http://www.cs.cornell.edu/courses/cs4780/2017sp/lectures/lecturenote11.html

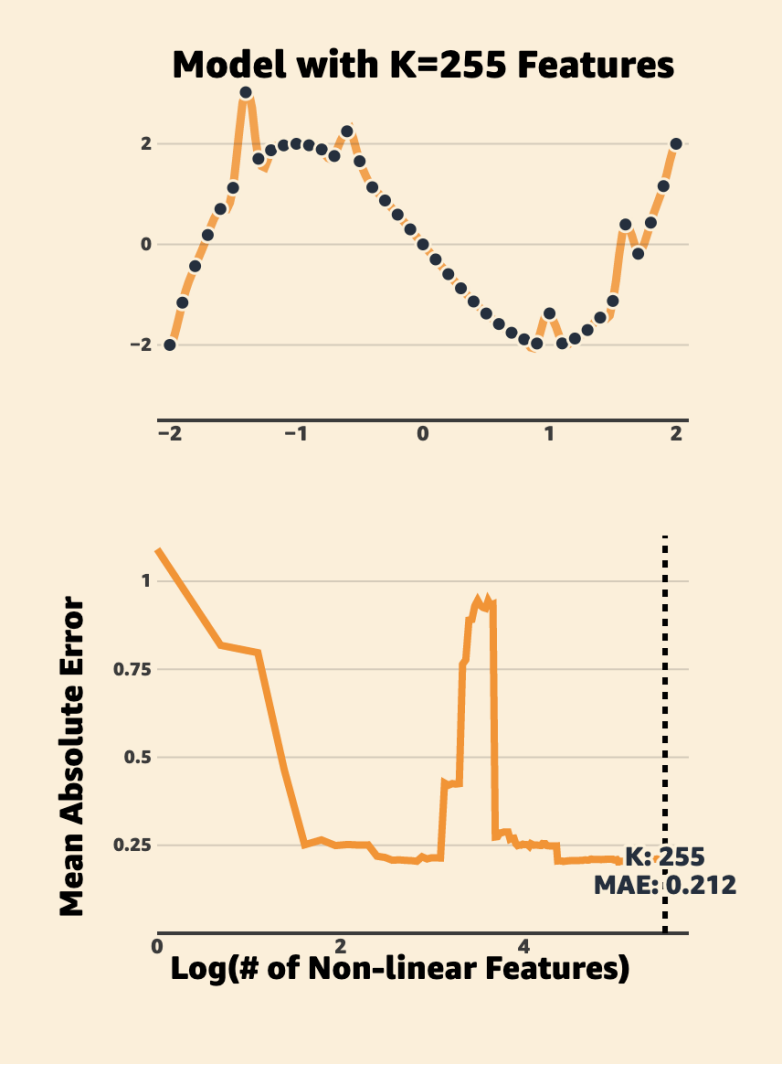
The double descent problem



https://openai.com/blog/deep-double-descent/

The double descent problem





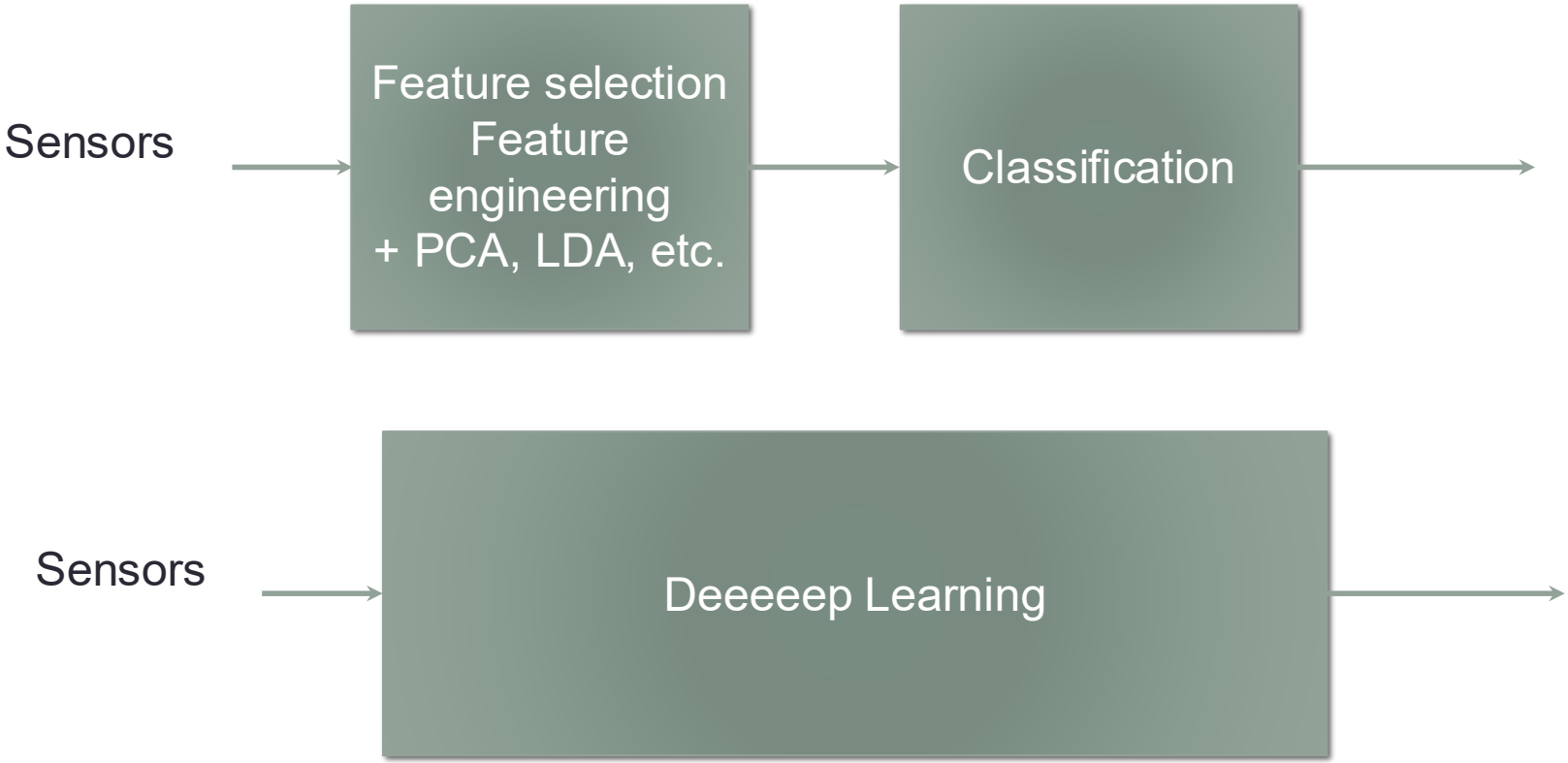
https://mlu-explain.github.io/double-descent/
https://mlu-explain.github.io/double-descent2/

Inductive bias and ML

- The **inductive bias** of a learning algorithm is a set of assumptions that the algorithm used to generalize to new inputs. http://www.cs.cmu.edu/~tom/pubs/NeedForBias_1980.pdf

- Your choice of model forces a certain type of behavior
  - Tells the model how to “overfit” the training data
- Putting inductive bias into deep learning model is easier than other models
  - ~~Domain knowledge to construct features~~
  - Domain knowledge to encourage certain learning behaviors

Traditional VS Deep learning

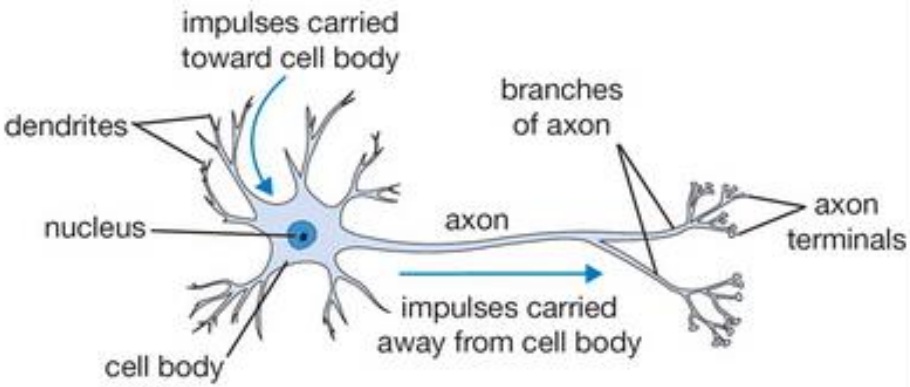


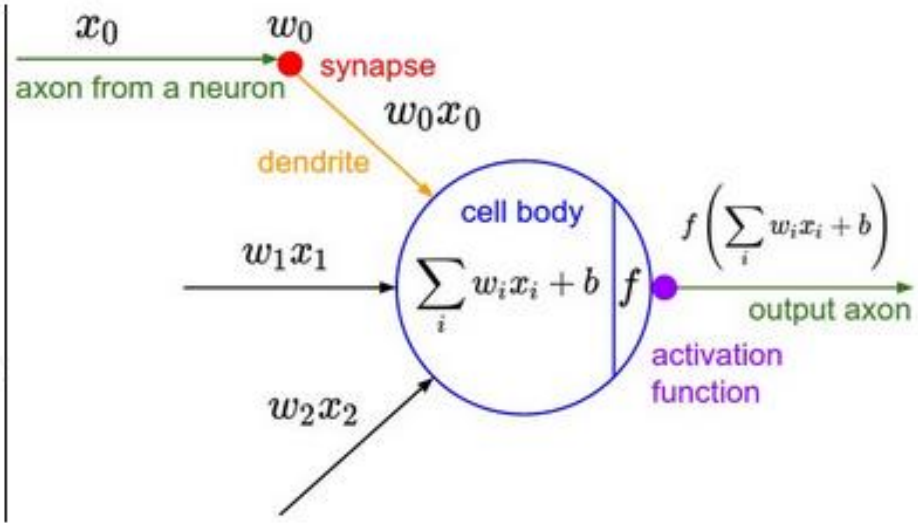
Neural networks

- Fully connected networks
  - Neuron
  - Non-linearity
  - Softmax layer
- DNN training
  - Loss function and regularization
  - SGD and backprop
  - Learning rate
  - Overfitting – dropout, batchnorm
- CNN, RNN, LSTM, GRU

Fully connected networks

- Many names: feed forward networks or deep neural networks or multilayer perceptron or artificial neural networks
- Composed of multiple neurons

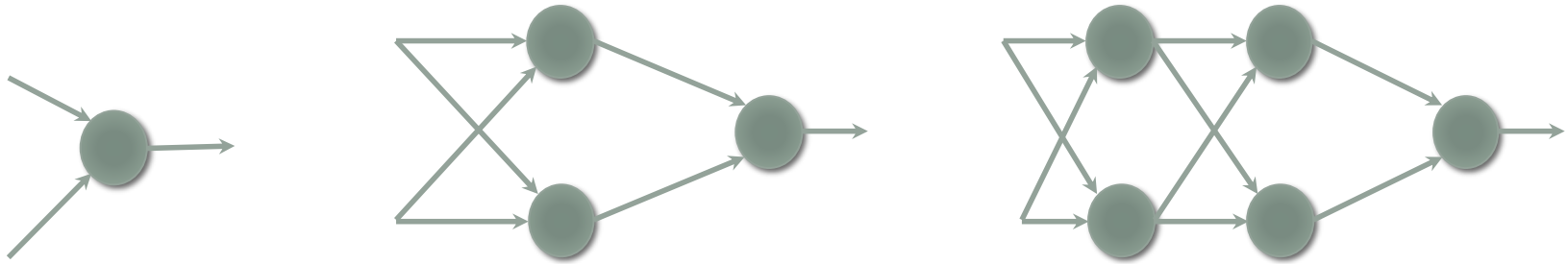


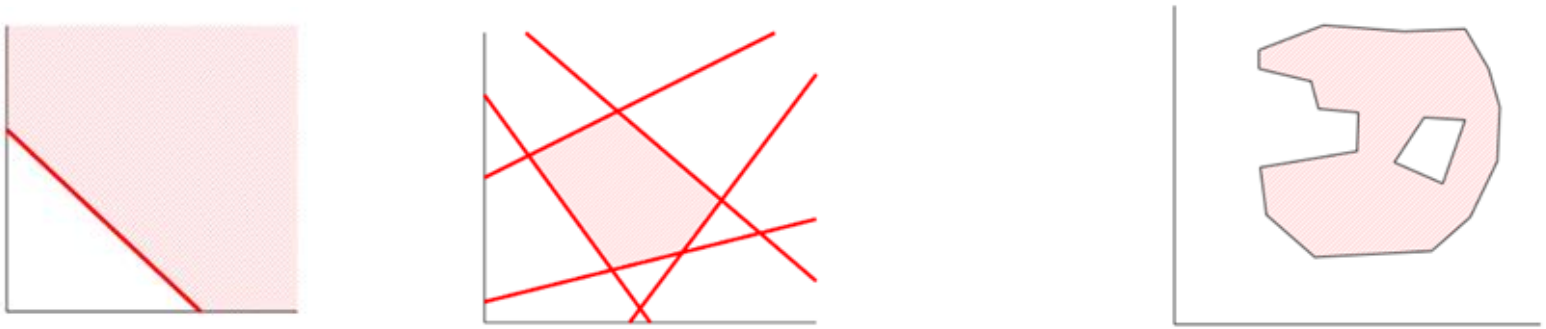


http://cs231n.github.io/neural-networks-1/

Combining neurons

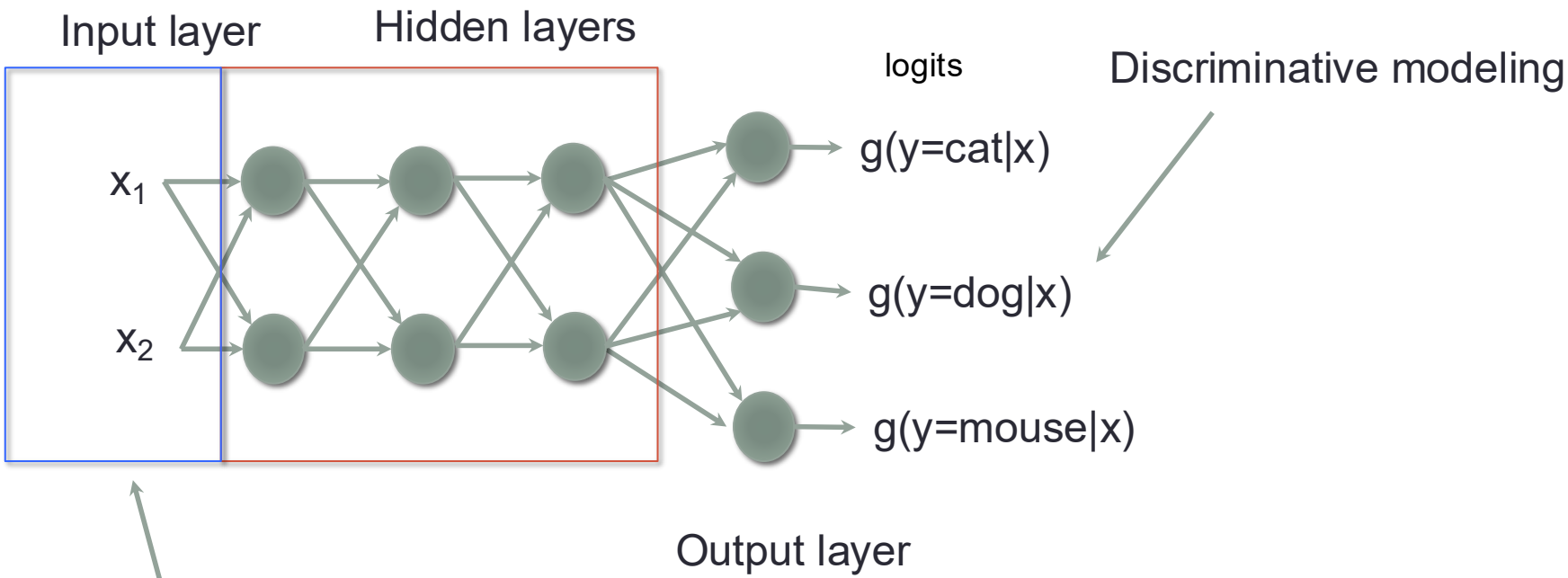
- Each neuron splits the feature space with a hyperplane
- Stacking neuron creates more complicated decision boundaries
- More powerful but prone to overfitting





Terminology

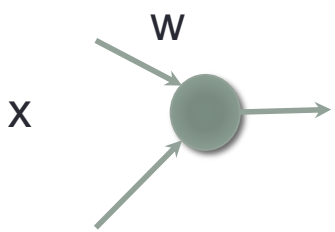
Deep in Deep neural networks means many hidden layers



Input should be scaled to have zero mean unit variance

Projections and Neural network weights

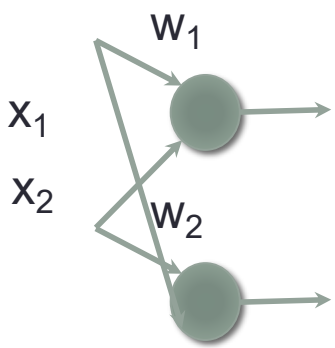
- $w^T x$

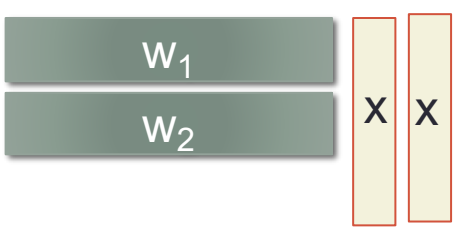


Projections and neural network weights

- $W^T[x_1, x_2]$

You can pack multiple inputs together to do one single matrix multiply

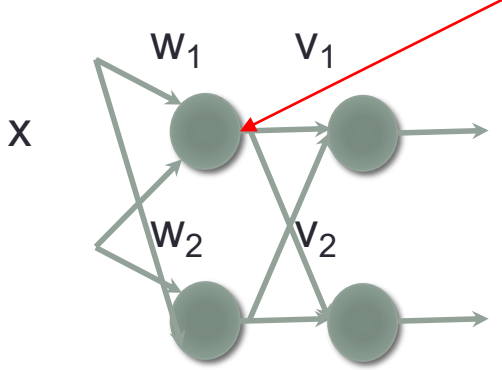




Neural network layer acts as nonlinear feature transform

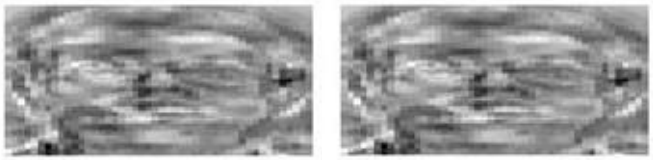
- $W^T x$

Without the nonlinearity the two matrices combine into one operation

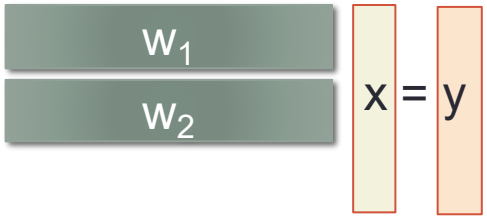


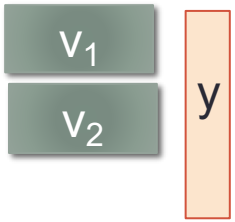
fischer projection = $V^T W^T x$
= $(WV)^T x$

Wv_1 Wv_2



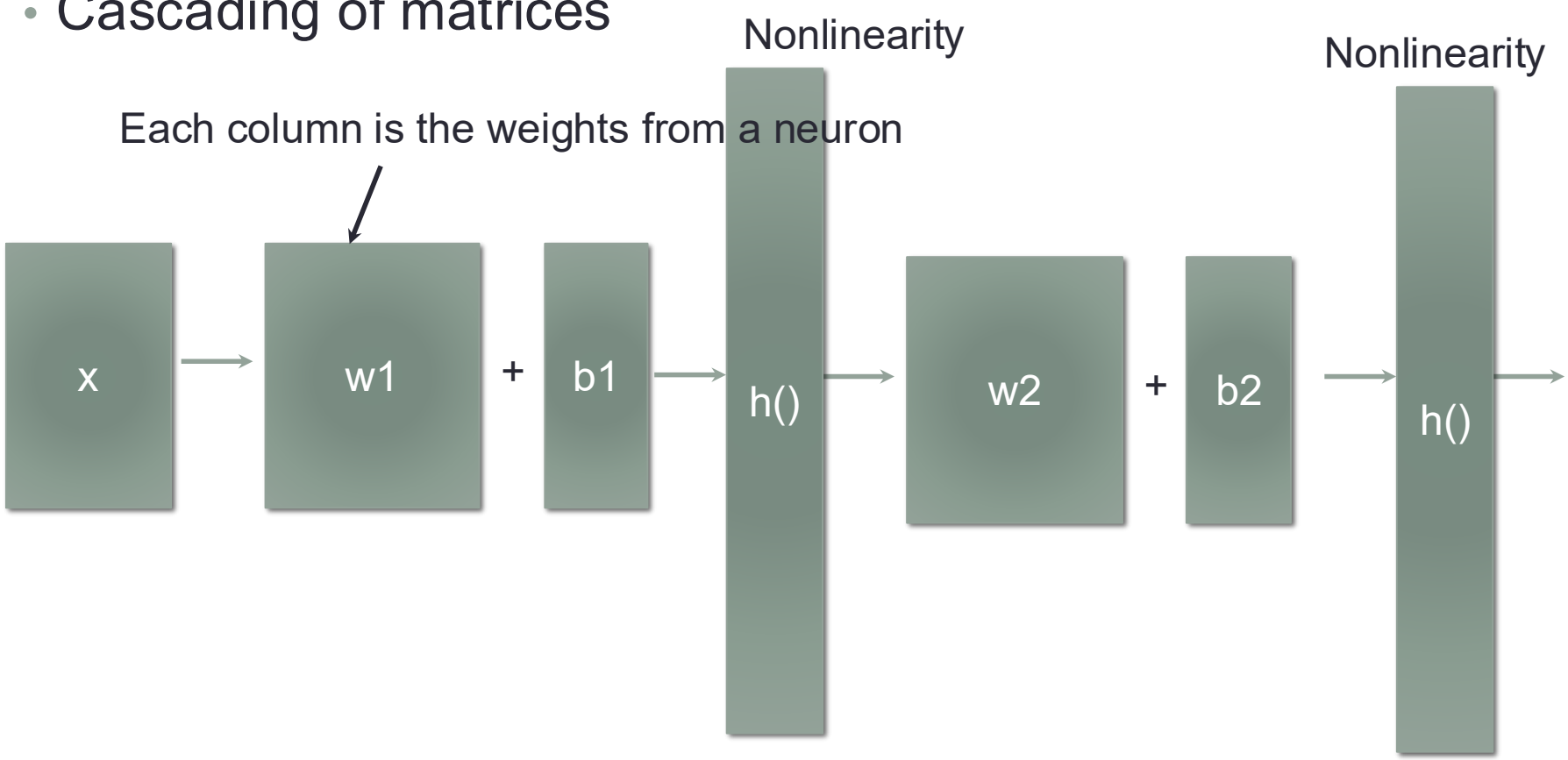
LDA projections





More linear algebra

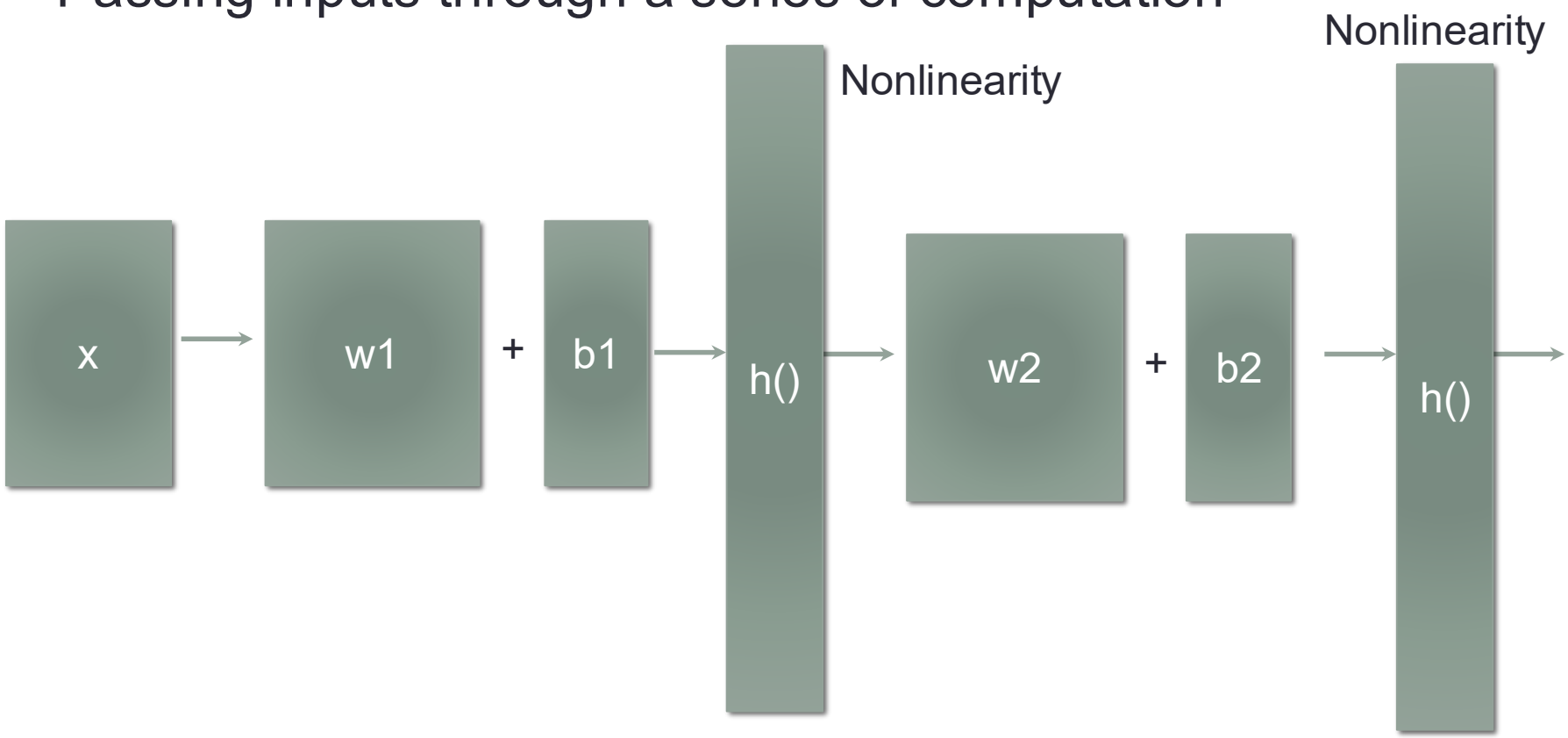
- Cascading of matrices



$h(W_2^T\, h(W_1^T X + \mathbf{b_1}) + \mathbf{b_2})$

Computation graph

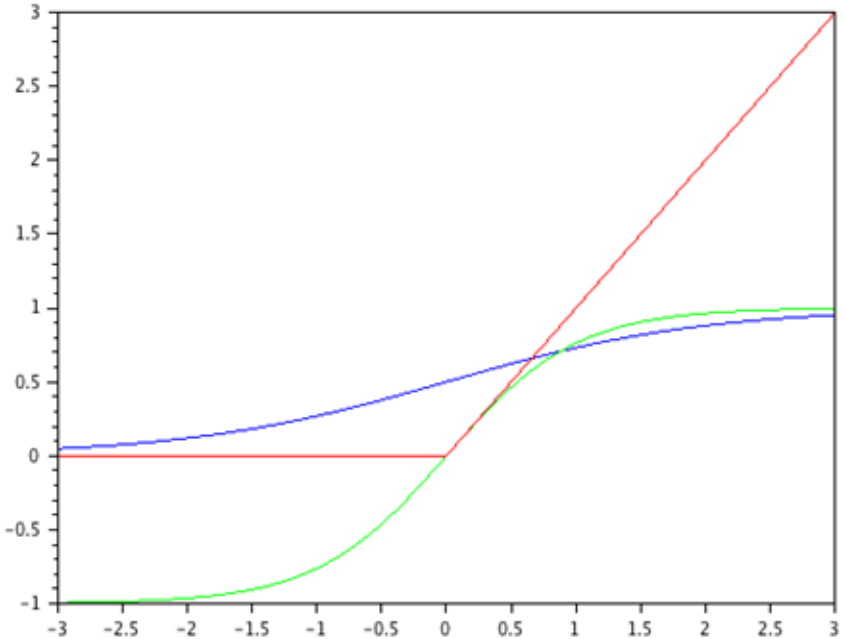
- Passing inputs through a series of computation



$h(W_2^T\, h(W_1^T X + \mathbf{b_1}) + \mathbf{b_2})$

Non-linearity

- The Non-linearity is important in order to stack neurons
- Sigmoid or logistic function
- tanh
- Rectified Linear Unit (ReLU)
  - LeakyReLU, ELU, PreLU
- Sigmoid Linear Units (SiLU)
  - Swish, Mish, GELU, SwiGLU
- Most popular is ReLU and its variants



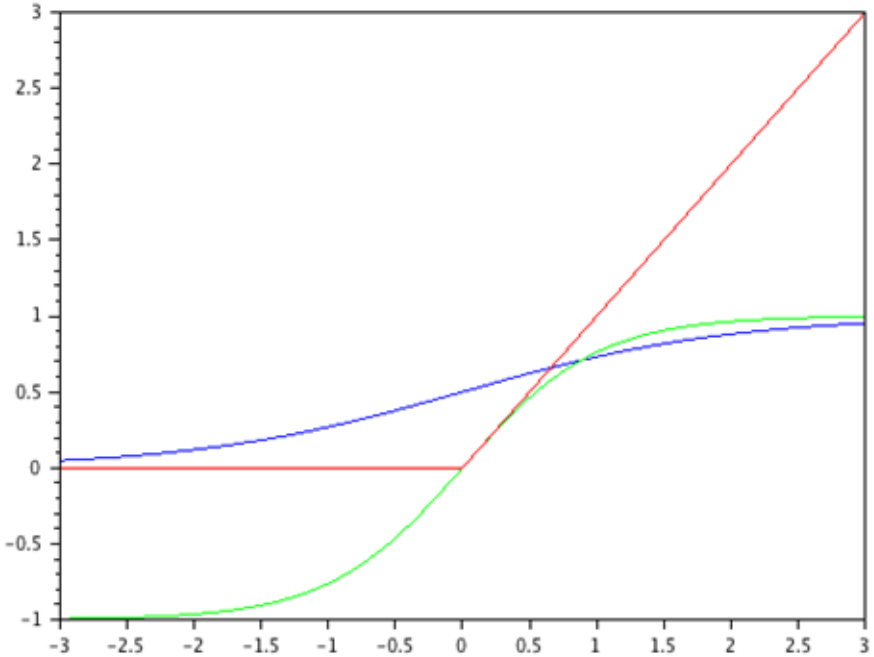
Non-linearity

- Sigmoid $$\frac{1}{1+e^{-x}}$$
- tanh

$$tanh(x)$$

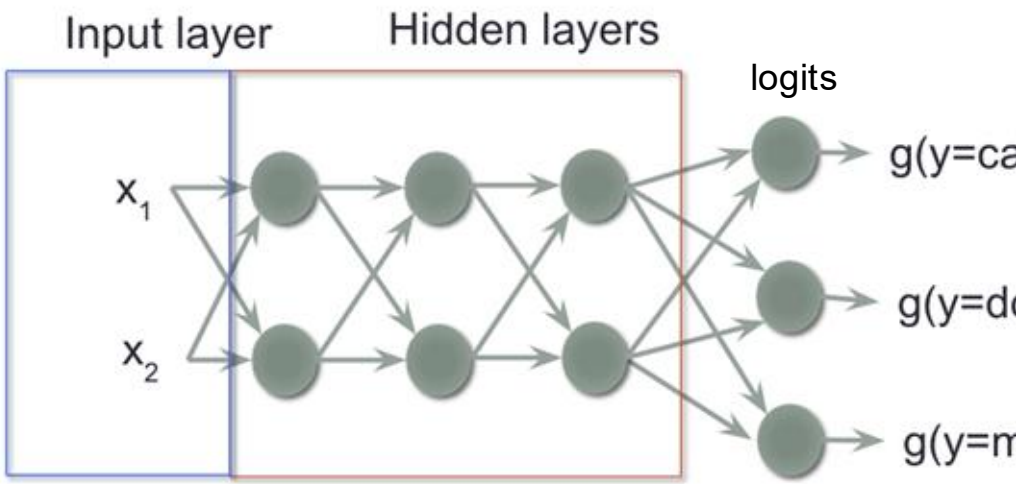
- Rectified Linear Unit (ReLU)

$$max(0, x)$$



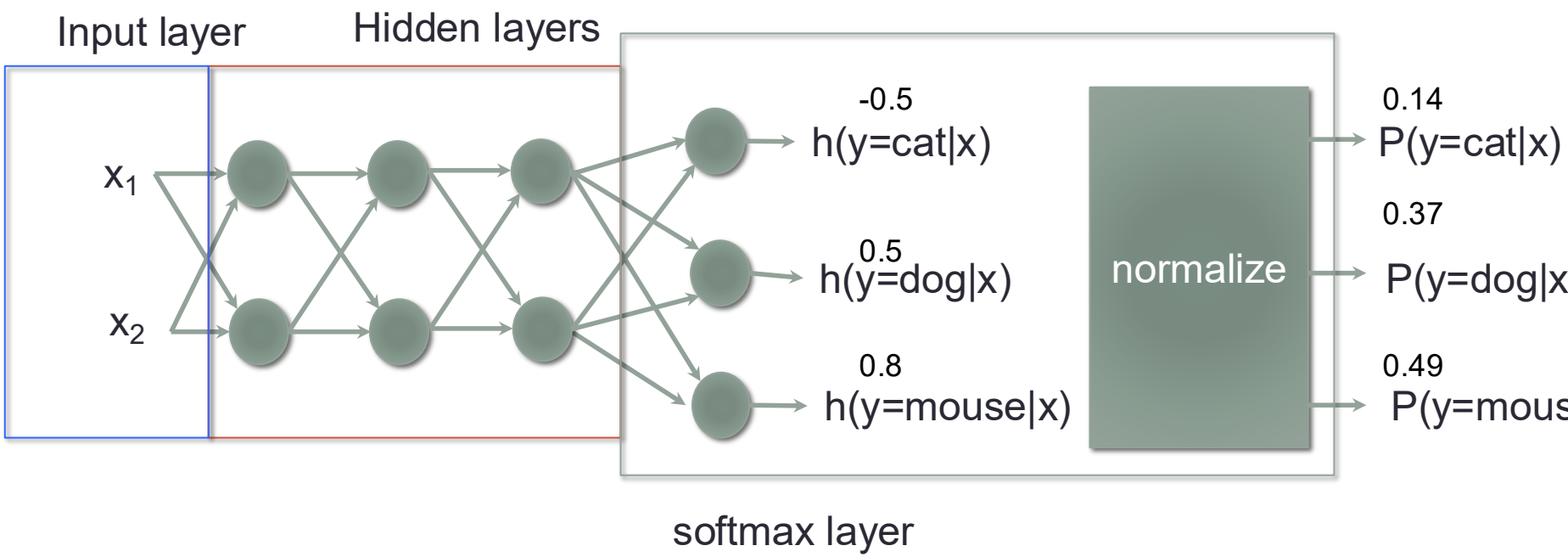
Output layer – Softmax layer

- We usually wants the output to mimic a probability function (0<=P<=1,sums to 1)
- Current setup has no such constraint
- The current output should have highest value for the correct class.
  - Value can be positive or negative number
- Takes the exponent
- Add a normalization



Softmax layer

$$P(y=j|x) = \frac{e^{h(y=j|x)}}{\Sigma_y e^{h(y|x)}}$$



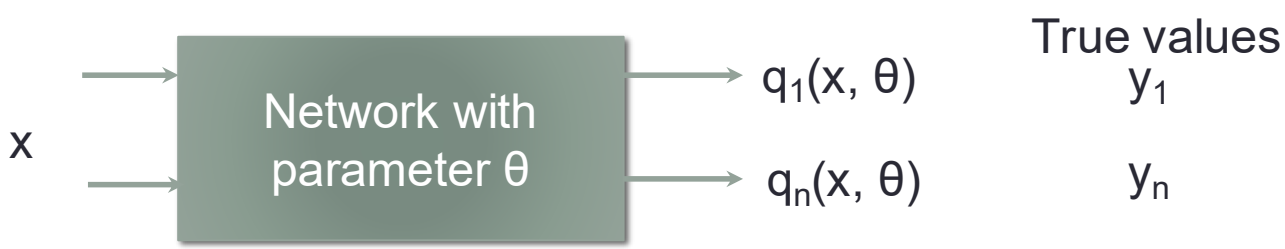
Neural networks

- Fully connected networks
  - Neuron
  - Non-linearity
  - Softmax layer
- DNN training
  - Loss function and regularization
  - SGD and backprop
  - Learning rate
  - Overfitting – dropout, batchnorm
- CNN, RNN, LSTM, GRU <- Next class

Objective function (Loss function)

- Can be any function that summarizes the performance into a single number
- Cross entropy

- Sum of squared errors



Cross entropy loss

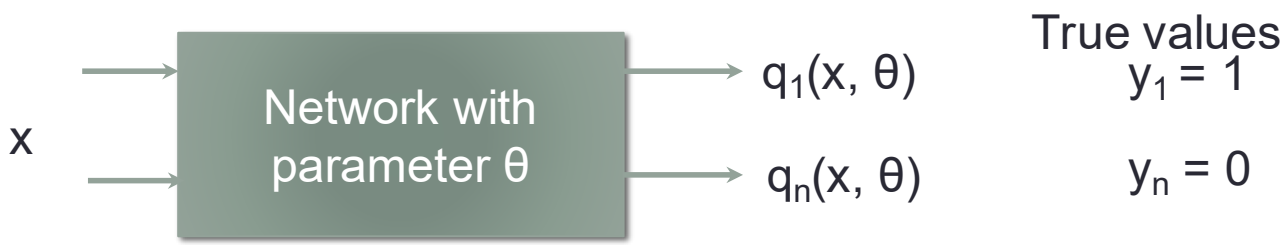
- Used for softmax outputs (probabilities), or classification tasks

Note Eq is calculated on one sample
Summed over whole batch for total loss

$$L = -\Sigma_n y_n log q_n(x, \theta)$$

- Where y_n is 1 if data x comes from class n
  0 otherwise

- L only has the term from the correct class
- L is non-negative with highest value when the output matches the true values, a "loss" function



Cross entropy loss & Logarithmic Loss (log loss)

- Minimizing the CE can be considered as the maximizing the log likelihood

$$L = -\Sigma_n y_n log q_n(x, \theta)$$

- Where y_n is 1 if data x comes from class n
  0 otherwise
- For binary class: $L(x_n) = \begin{cases} -\log(h(x_n)) & \text{, if } y_n = 1 \\ -\log(1-h(x_n)) & \text{, if } y_n = 0 \end{cases}$

$$L = [y_n log(h(x_n))] + [(1 - y_n) log(1 - h(x_n))]$$

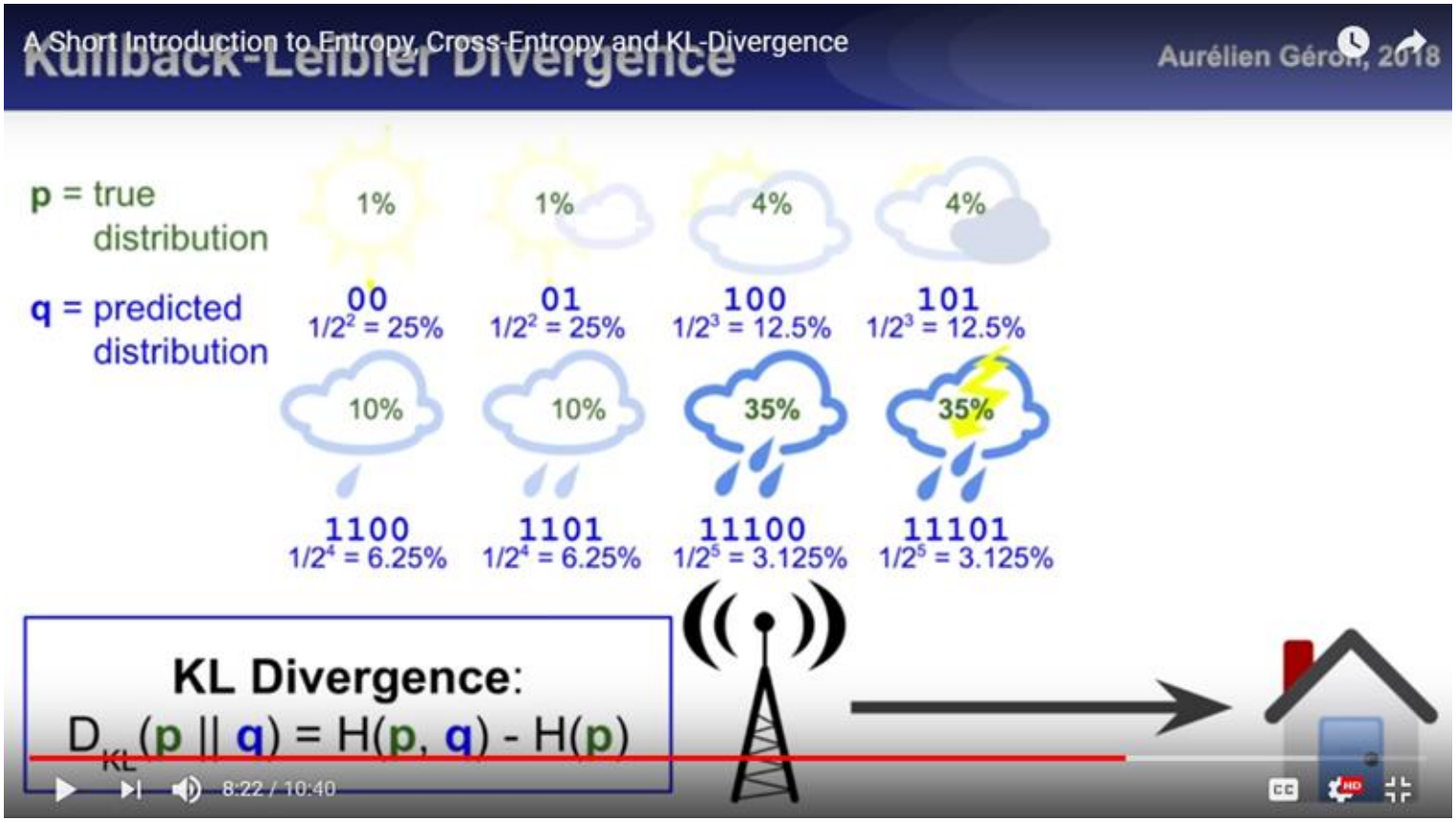
Same as log likelihood of logistic regression
Negative in front because we are minimizing the loss vs maximizing the probability

$$p(y \mid x; \theta) = (h_\theta(x))^y (1 - h_\theta(x))^{1-y}$$

Other views of CE loss

Relationship between Entropy, CE, and KL Divergence

https://www.youtube.com/watch?v=ErfnhcEV1O8



Notes on CE loss

- If classes is ordered or errors are not equally bad, it might not be ideal
  - Class 0: Perfect, Class 1: Good, Class 2: Average, Class 3: Bad
    - Groundtruth [1, 0, 0, 0] (Perfect)
    - Model A [0.5, 0.0, 0.1, 0.4] Model B [0.5, 0.4, 0.1, 0.0]
  - Soln: use Squared EMD loss https://arxiv.org/abs/1611.05916
- You can have better loss but worse accuracy
  - Groundtruth [1, 0, 0].
    - Model A [0.4, 0.3, 0.3], Model B [0.45, 0.5, 0.05]
  - Soln: monitor accuracy as well as loss
- Leads to overconfidence
  - Soln: label smoothing, https://paperswithcode.com/method/label-smoothing, Calibration https://arxiv.org/abs/1706.04599

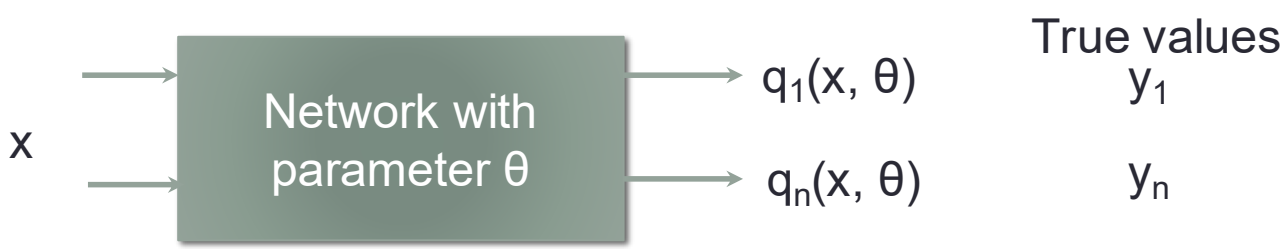
Sum of squared errors (MSE)

- Used for any real valued outputs such as regression

$$L = \frac{1}{2}\Sigma_n (y_n - q_n(x, \theta))^2$$

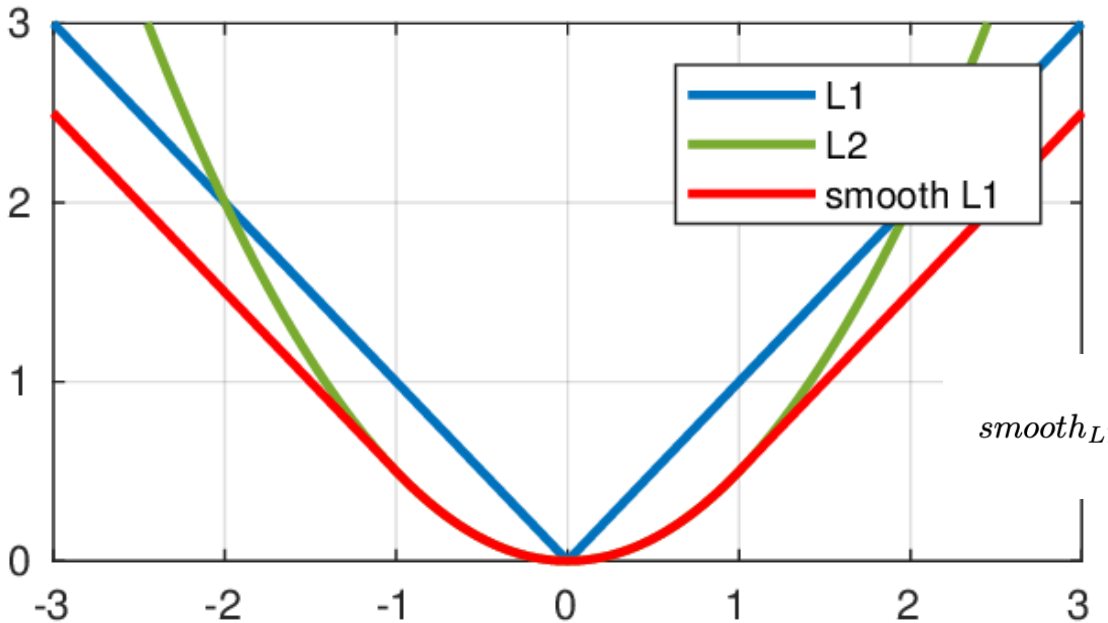
n is output dimension index
Note Eq is calculated on one sample
Summed over whole batch for total loss

- Nonnegative, the better the lower the loss



Notes on MSE loss

- L1 vs L2 loss
  - L1 robust to outliers
  - L2 easier to optimize (smooth gradient)
- Smoothed L1 combination of L1 and L2

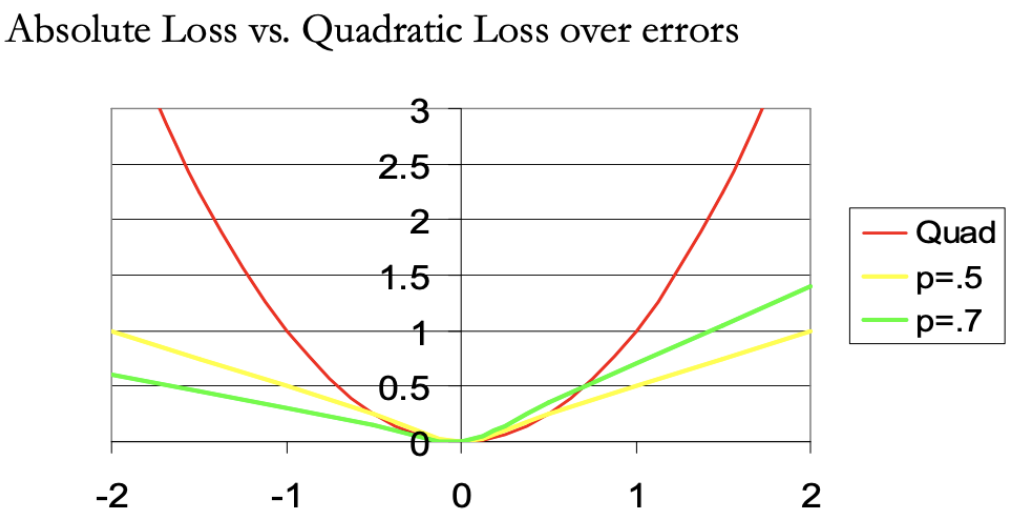


$$smooth_{L1}(x) = \begin{cases} \frac{1}{2}x^2 & \text{if } |x| < 1 \\ |x| - \frac{1}{2} & \text{otherwise} \end{cases}, \quad (4)$$

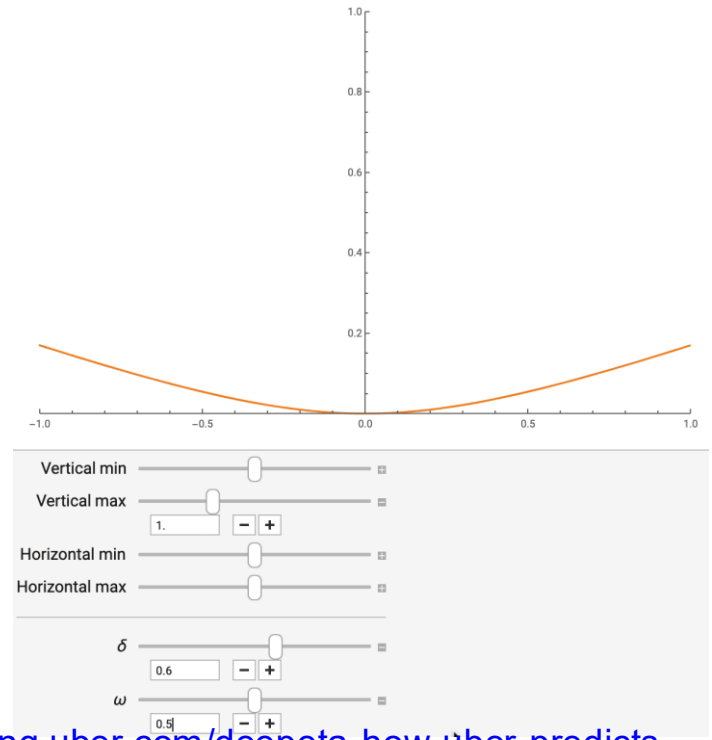
https://www.researchgate.net/publication/321180616_Wing_Loss_for_Robust_Facial_Landmark_Localisation_with_Convolutional_Neural_Networks

Notes on MSE loss

- Loss weights underestimate and overestimate to be the same
  Soln: asymetric loss (Quantile loss – L1, Huber loss – smoothed L1)



https://www.bauer.uh.edu/rsusmel/phd/ec1-25.pdf



https://eng.uber.com/deepeta-how-uber-predicts-arrival-times/

Regularization

Regularization: trading model bias for model variance

There two main approaches to regularize neural networks

- Explicit regularization
  - Deals with the loss function
- Implicit regularization
  - Deals with the network

Famous types of regularization

- L1 regularization: Regularizing term is a sum

$$\frac{1}{2}\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x_i})^2 \; + \Sigma_j \mid \theta_j \mid$$

- L2 regularization: Regularizing term is a sum of squares

$$\frac{1}{2}\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x_i})^2 \; + \Sigma_j \mid \theta_j \mid^2$$

(Not to be confused with L1 and L2 losses)

http://www.chioka.in/differences-between-l1-and-l2-as-loss-function-and-regularization/

Numerical example

Training data x = [3, 2, 1] y = 10, regression task

Objective: $10 = w_1*3 + w_2*2 + w_3*1$ Find w_1, w_2, w_3

w_1	w_2	w_3	L1 reg loss	L2 reg loss
3	0.25	0.5	3.75	9.31
5	-2	-1	8	30
3.33	0	0	3.33	11.11
2.14	1.42	0.71	4.29	7.14

L1 does feature selection (makes most numbers 0) - can be used to do feature selection
L2 spreads the numbers (no 0) - gives generalization

L1 L2 regularization notes

- Can use both at the same time
  - People claim L2 is superior
  - Weight decay can be considered as L2 regularization (under simple optimization techniques)
    - https://arxiv.org/abs/1711.05101
- I found them useless in practice for deep neural networks
  - Works when data is small (transfer learning)
- Other regularization methods exist
    - Ex Weight Decay

Weight decay

- A regularization technique popular in deep learning models
- Every update step, each weight loses a portion of their value.

$$\theta_t = (1 - \lambda')\theta_{t-1} - \eta \frac{\partial L(\theta_{t-1})}{\partial \theta}$$

- Can be considered as L2 regularization in special cases (fixed learning rate).

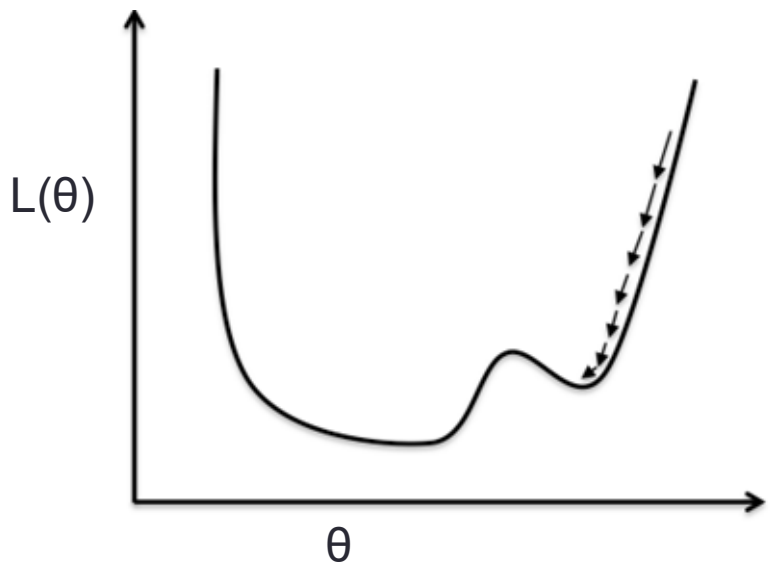
https://openreview.net/pdf?id=J7V_4aauV6B

Neural networks

- Fully connected networks
  - Neuron
  - Non-linearity
  - Softmax layer
- DNN training
  - Loss function and regularization
  - SGD and backprop
  - Learning rate
  - Overfitting – dropout, batchnorm
- CNN, RNN, LSTM, GRU

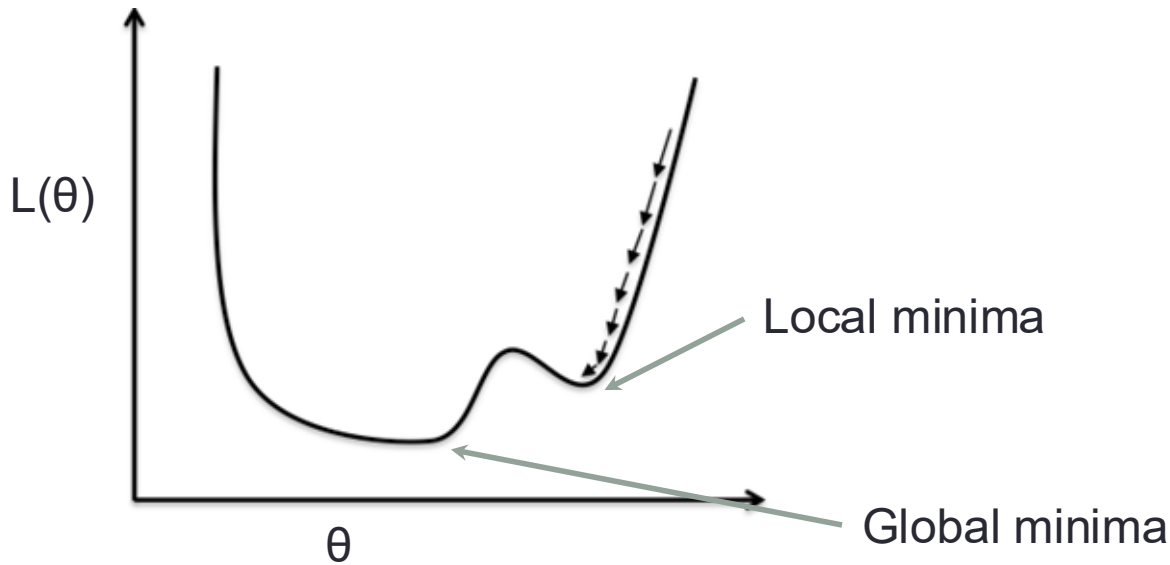
Minimization using gradient descent

- We want to minimize L with respect to θ (weights and biases)
  - Differentiate with respect to θ
  - Gradients passes through the network by Back Propagation



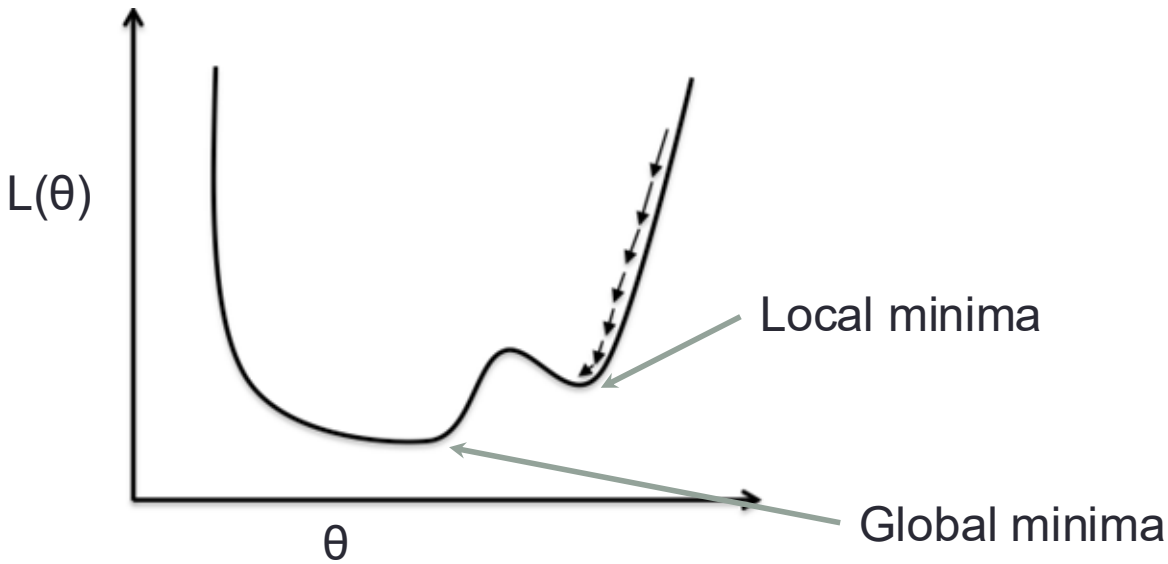
Deep vs Shallow

- The loss function of neural network is non-convex (and non-concave)
  - Local minimas can be avoided with convexity
    - Linear regression, SVM are convex optimization
  - Convexity gives easier training
    - Does not imply anything about the generalization of the model
    - The loss is optimized by the training set



Deep vs Shallow

- If deep, most local minimas are the global minima!
  - Always a way to lower the loss in the network with millions of parameters
  - Enough parameters to remember every training examples
  - Does not imply anything about generalization



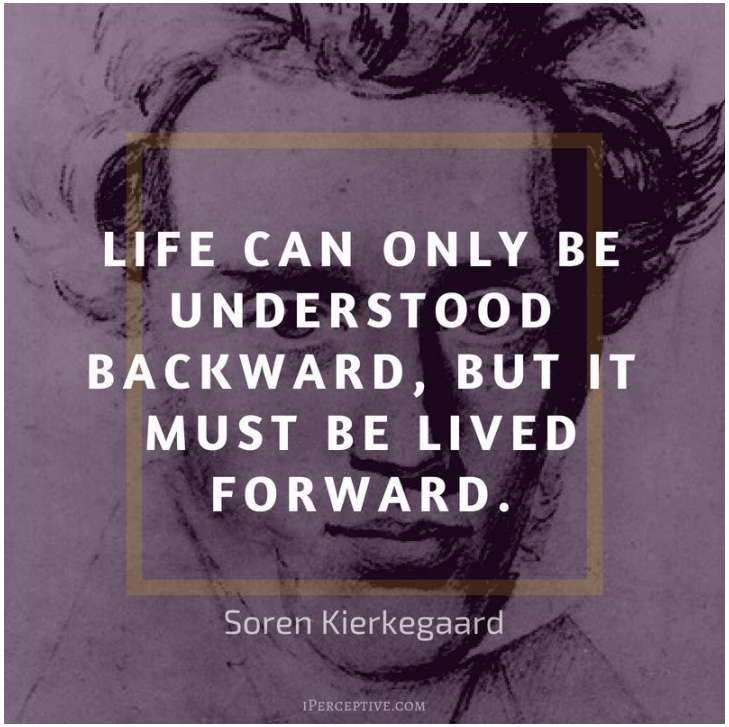
Kenji Kawaguchi,Deep Learning without Poor Local Minima, 2016 https://arxiv.org/abs/1605.07110

Differentiating a neural network model

- We want to minimize loss by gradient descent
- A model is very complex and have many layers! How do we differentiate this!!?

Back propagation

- Forward pass
  - Pass the value of the input until the end of the network
- Backward pass
  - Compute the gradient starting from the end and passing down gradients using chain rule



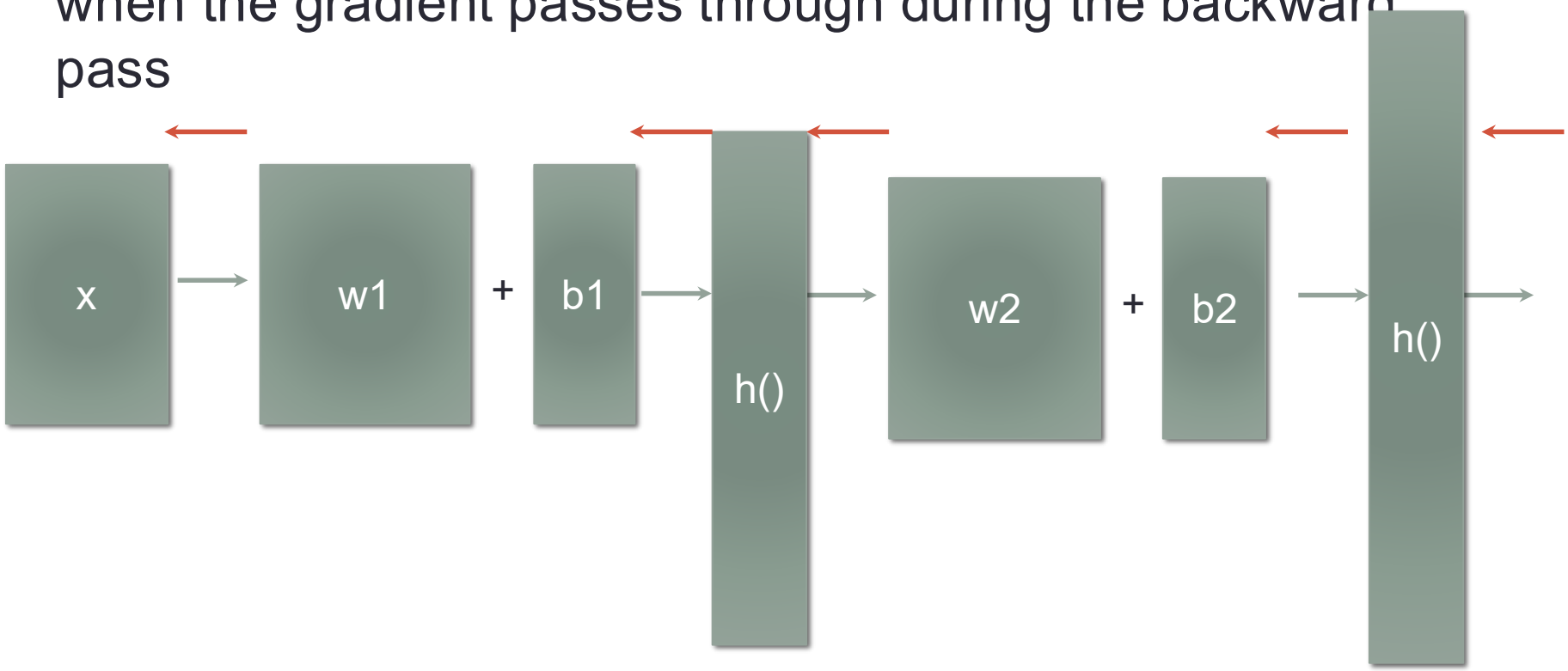
Examples to read

https://alonalj.github.io/2016/12/10/What-is-Backpropagation/

https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backprop and computation graph

- We can also define what happens to a computing graph when the gradient passes through during the backward pass



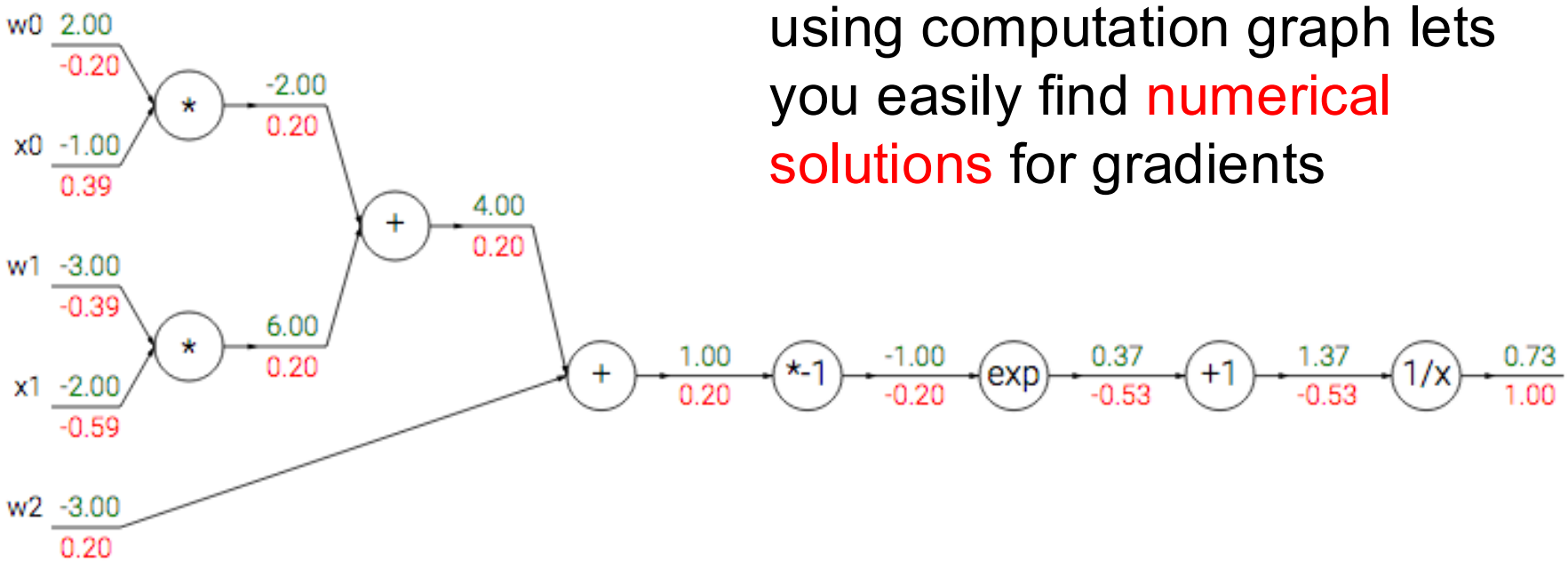
This lets us to build any neural networks without having to redo all the derivation as long as we define a forward and backward computation for the block.

Numerical gradient flow

- Let's find the gradient of

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

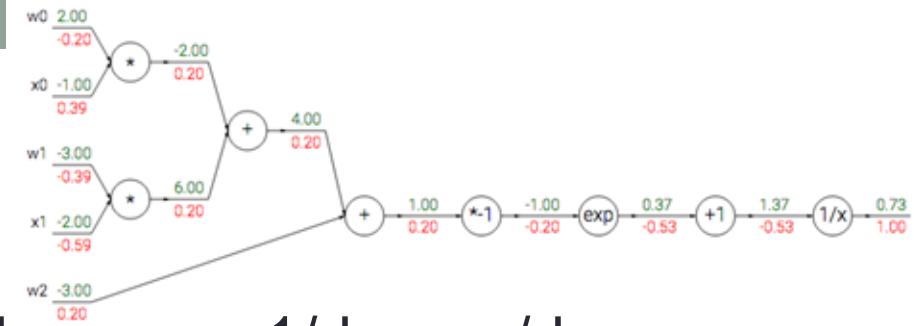
Computation graph



Doing backprop (chain rule) by using computation graph lets you easily find numerical solutions for gradients

http://cs231n.github.io/optimization-2/

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

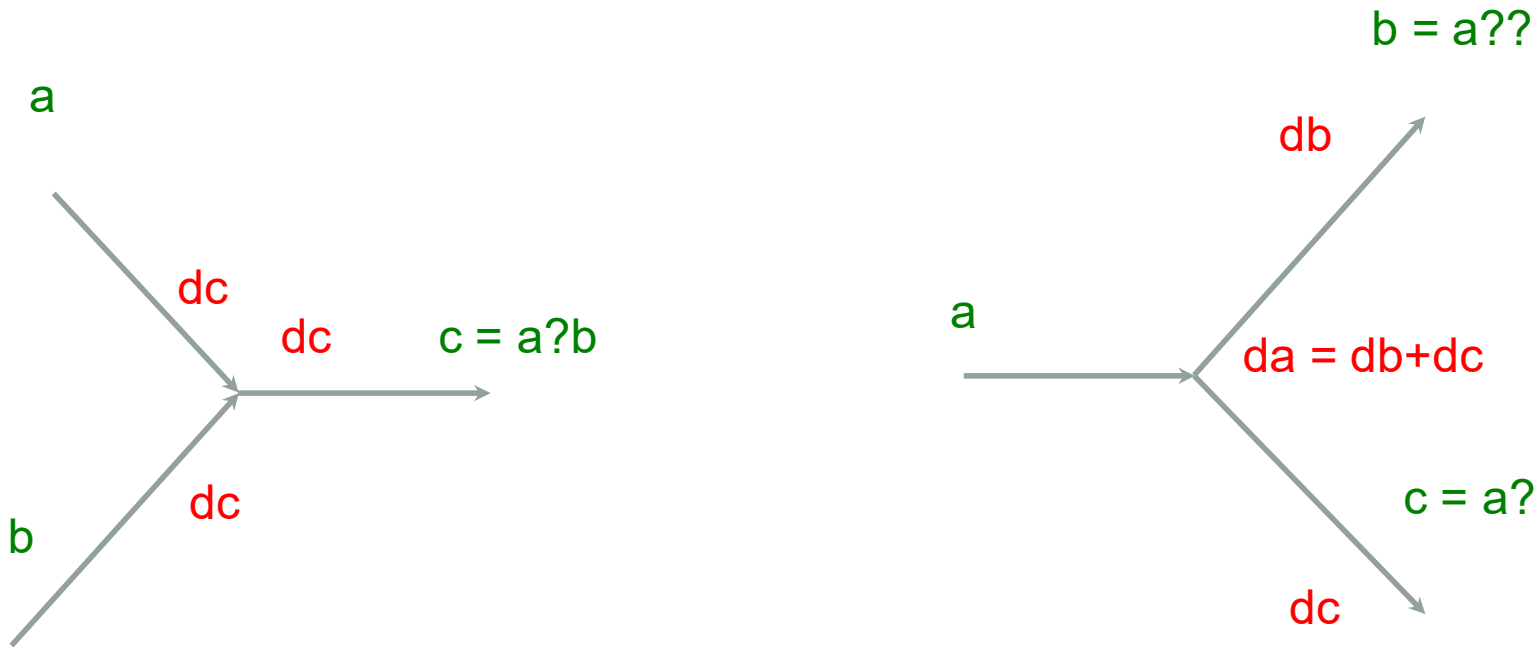


- w = [0, -3, -3]
- x = [-1, -2]
- t_0 = w[0]*x[0]
- t_1 = w[1]*x[1]
- t_{01} = t_0+t_1
- t_{012} = t_{01}+w[2]
- n_t = -t_{012}
- e = exp(n_t)
- denom = e+1
- f = 1/denom

- ddenom = -1/denom/denom
- de = 1*ddenom
- dn_t = exp(n_t)*de
- dt_{012} = -dn_t
- dw_2 = 1*dt_{012}
- dt_{01} = 1*dt_{012}
- dt_0 = 1*dt_{01}
- dt_1 = 1*dt_{01}
- dw_1 = x[1]dt_1
- dx_1 = w[1]dt_1
- dw_0 = x[0]dt_0; dx_0 = w[0]dt_0

Perform backward pass in reverse order. No need to explicitly find overall derivative
This technique is called autograd. Many frameworks use it today.

Gradient flow at forks



Forward and backward pass acts differently at forks

Gradient and non-linearities

We can now talk about how good a non-linearity is by looking at the gradients.

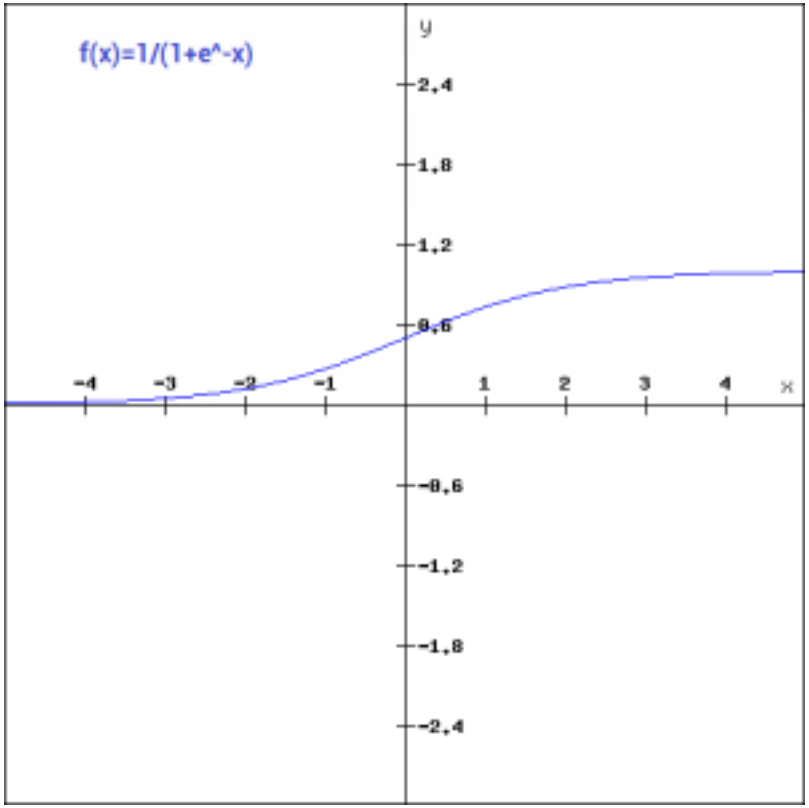
We want

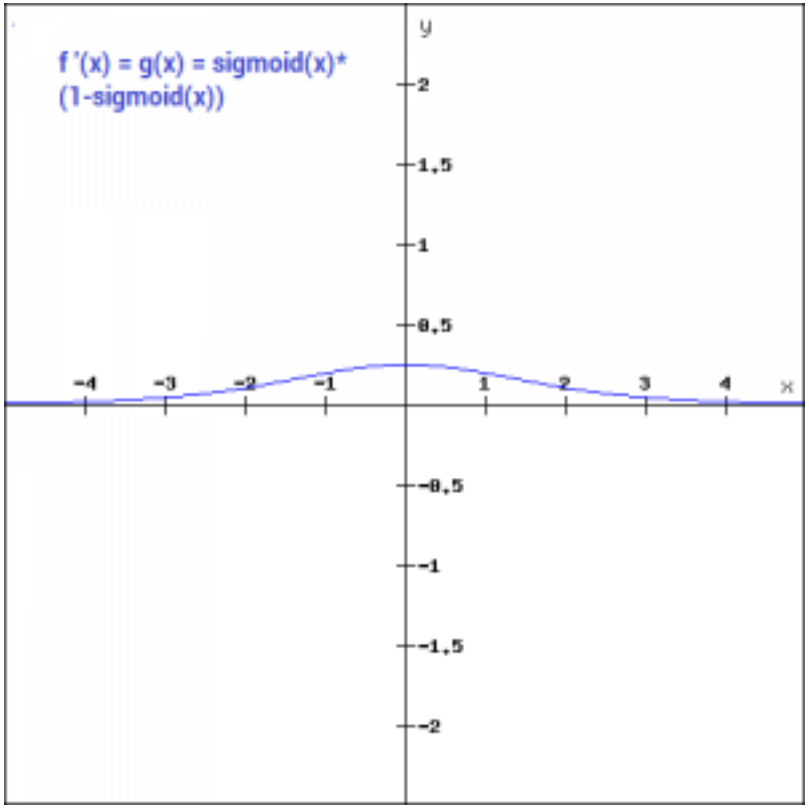
- Something that is differentiable numerically
- Cheap to compute
- Big gradients at every point

Notes on non-linearity

- Sigmoid

  Models get stuck if fall go far away from 0. Output always positive



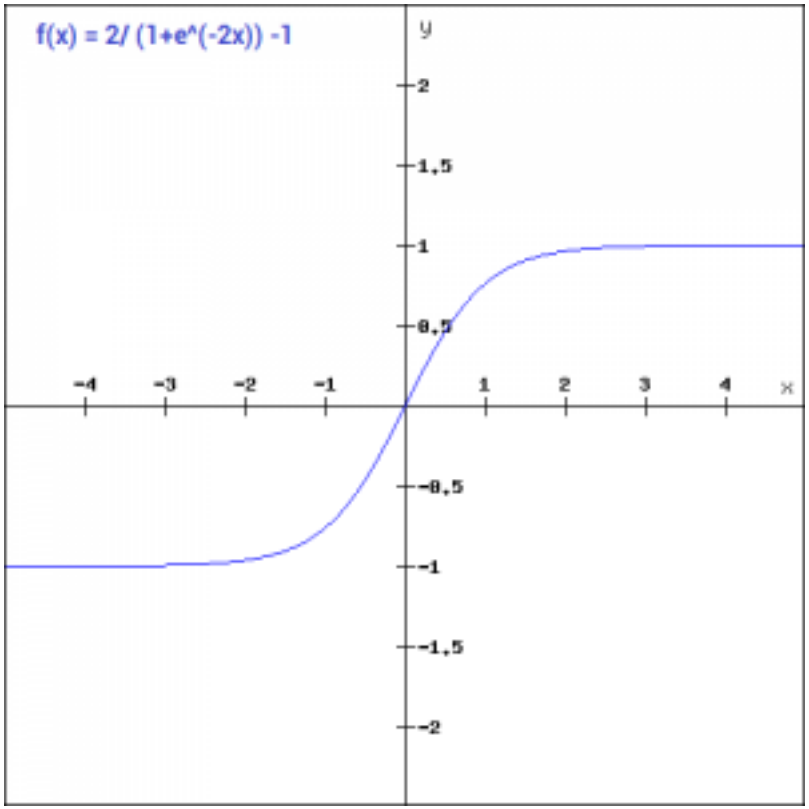


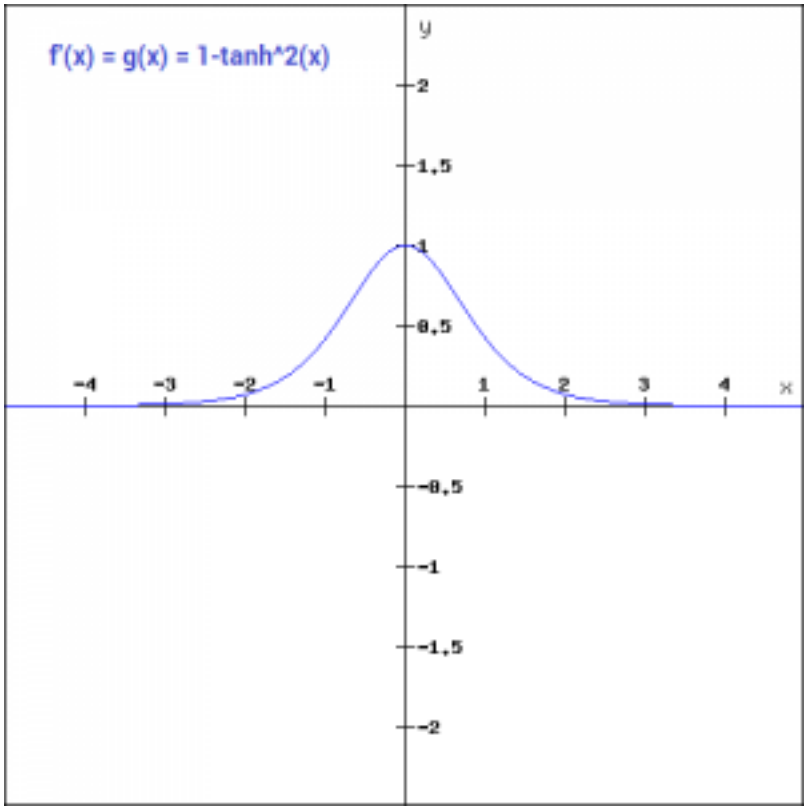
https://www.analyticsvidhya.com/blog/2017/10/fundamentals-deep-learning-activation-functions-when-to-use-them/

Notes on non-linearity

- Tanh

Output can be +-. Models get stuck if far away from 0



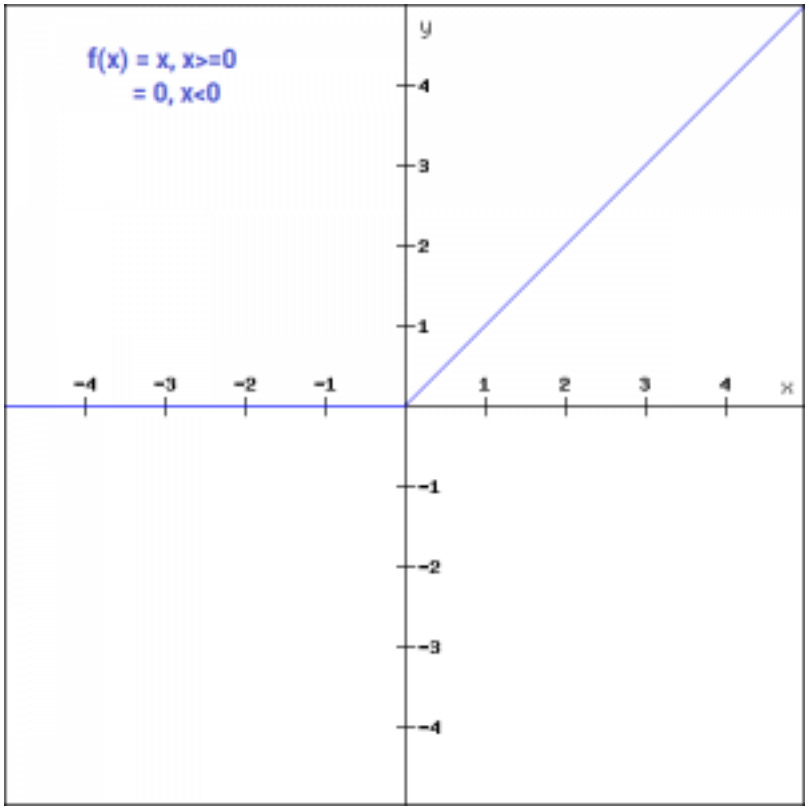


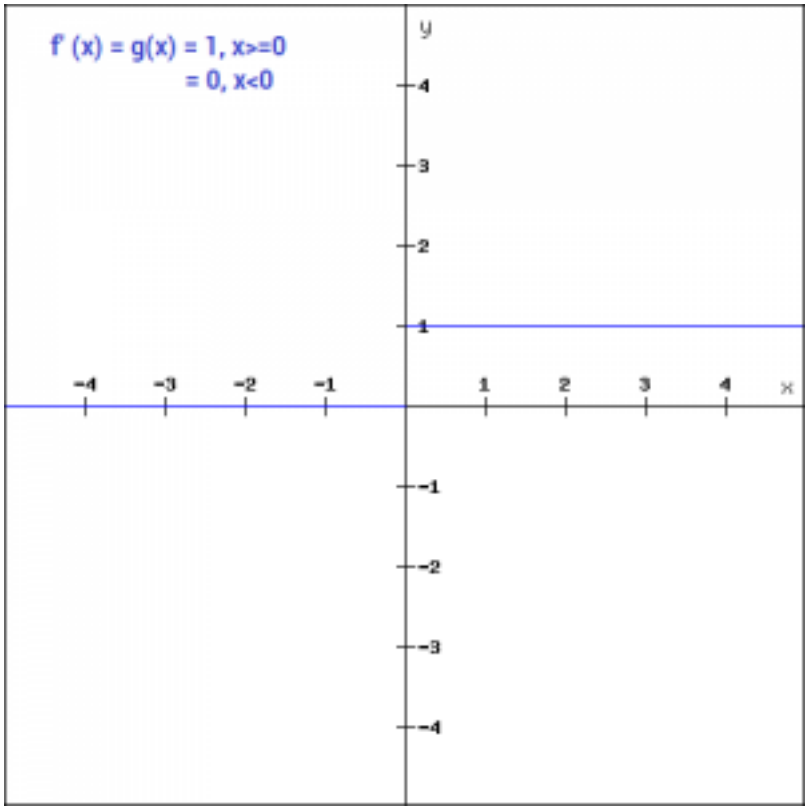
https://www.analyticsvidhya.com/blog/2017/10/fundamentals-deep-learning-activation-functions-when-to-use-them/

Notes on non-linearity

- ReLU

  High gradient in positive. Fast compute. Gradient doesn't move in negative



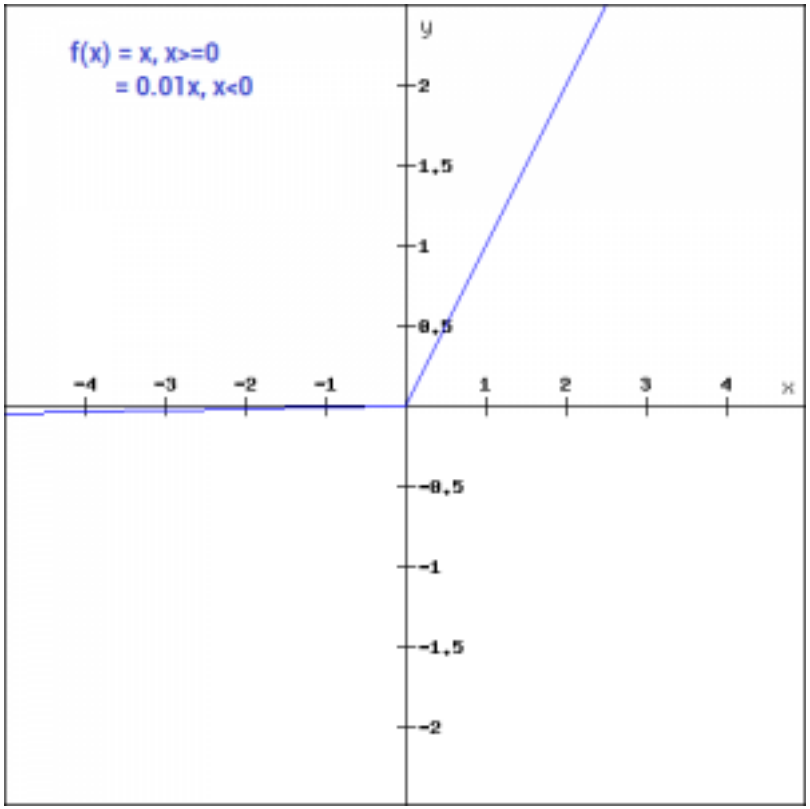


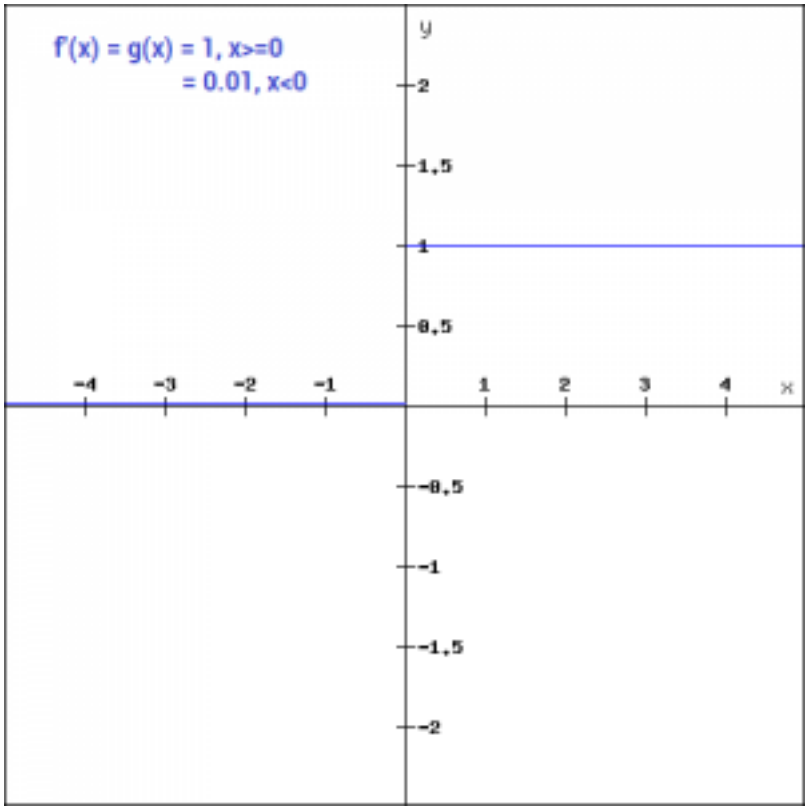
https://www.analyticsvidhya.com/blog/2017/10/fundamentals-deep-learning-activation-functions-when-to-use-them/

Notes on non-linearity

- Leaky ReLU

  Negative part now have some gradient. Small improvements depending on tasks





https://www.analyticsvidhya.com/blog/2017/10/fundamentals-deep-learning-activation-functions-when-to-use-them/

Notes on non-linearity

- Leaky ReLU
  Fixed slope
- PreLU
  Slope is learnable
  Different layer can have difference slope

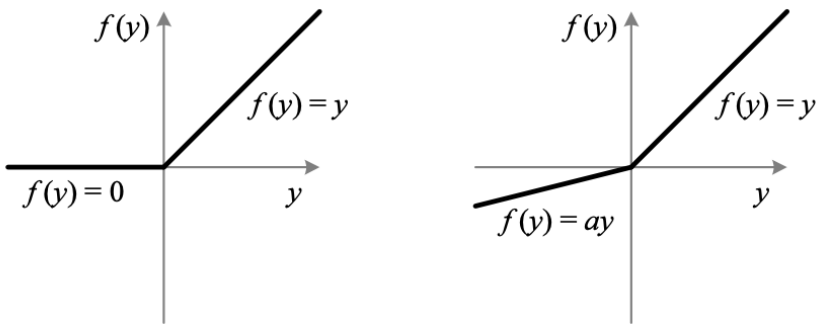


Figure 1. ReLU vs. PReLU. For PReLU, the coefficient of the negative part is not constant and is adaptively learned.

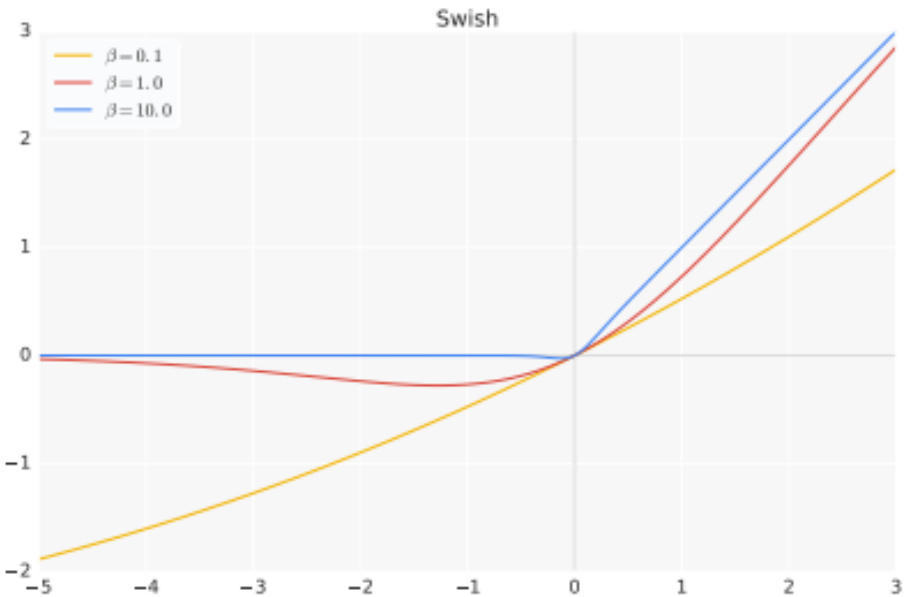
https://paperswithcode.com/paper/delving-deep-into-rectifiers-surpassing-human

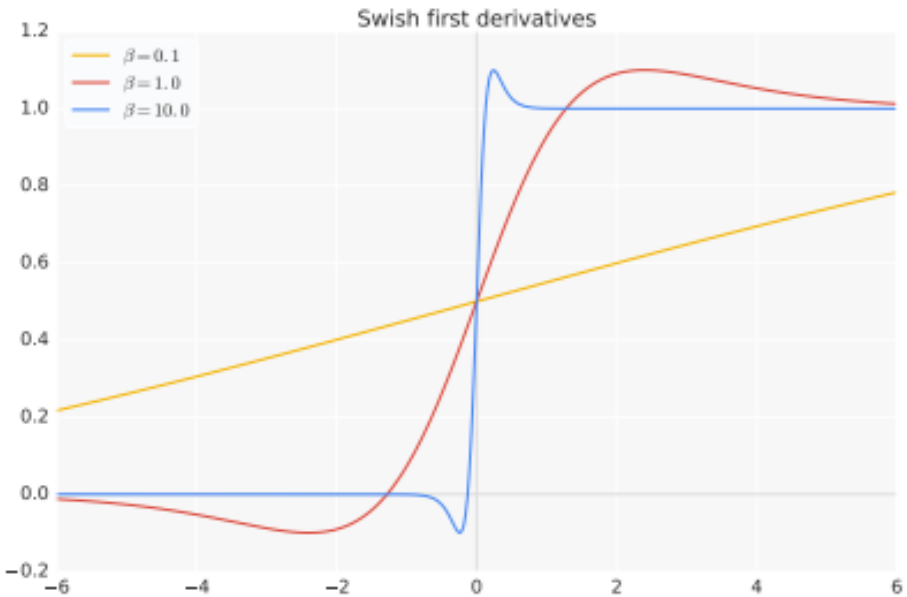
Notes on non-linearity

$$x \cdot sig(\beta x)$$

- Swish

  Nonnegative everywhere. Not monotonic. Learnable Beta or set to 1





[Searching for Activation Functions - arXiv]

SiLU, GELU

SiLU

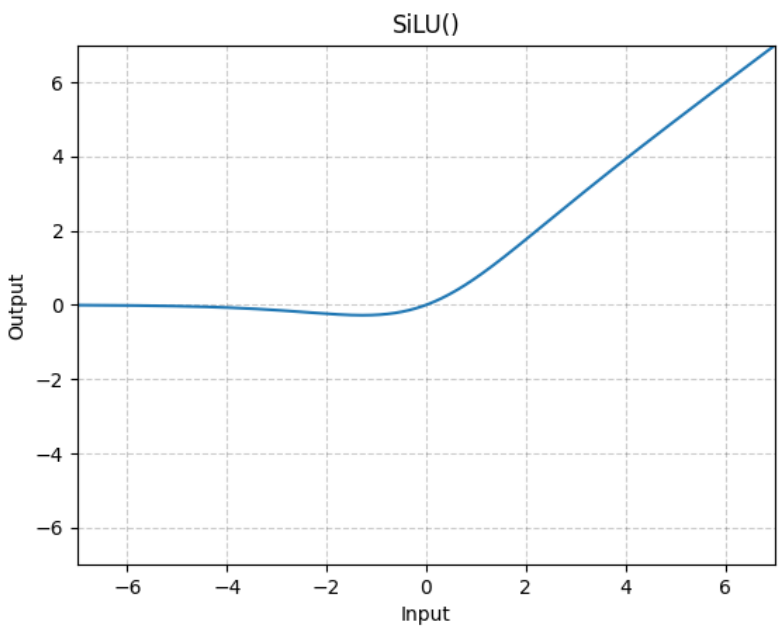
(or Swish – Swish paper comes after SiLU but is more popular)

CLASS torch.nn.SiLU(*inplace=False*) [SOURCE]

Applies the Sigmoid Linear Unit (SiLU) function, element-wise.

The SiLU function is also known as the swish function.

$$\text{silu}(x) = x * \sigma(x), \text{where } \sigma(x) \text{ is the logistic sigmoid.}$$



GELU

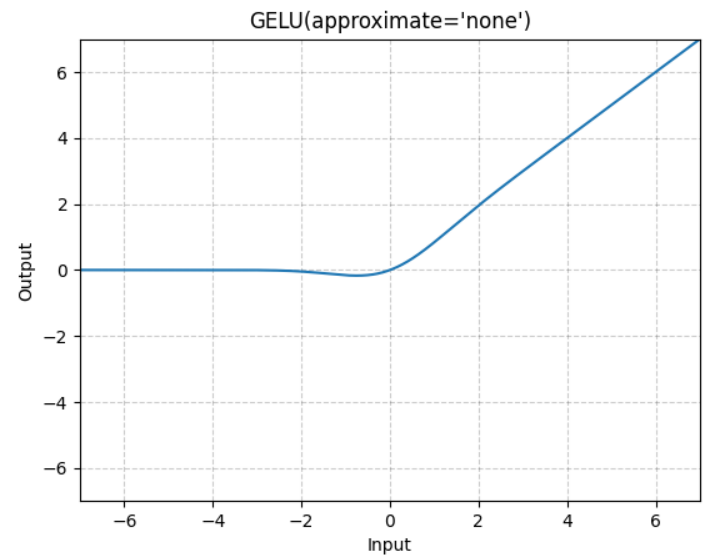
CLASS torch.nn.GELU(*approximate='none'*) [SOURCE]

Applies the Gaussian Error Linear Units function.

$$\text{GELU}(x) = x * \Phi(x)$$

where $\Phi(x)$ is the Cumulative Distribution Function for Gaussian Distribution.

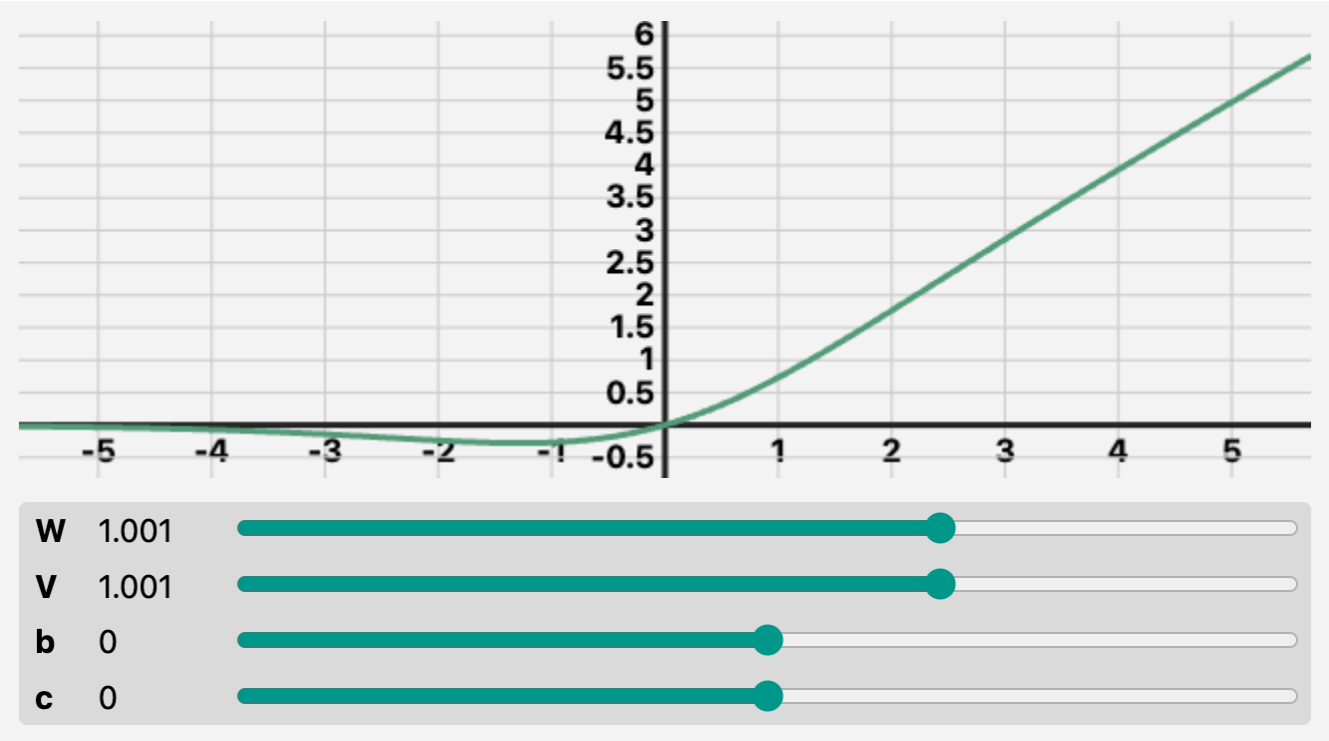
Set beta to 1.702 (approximation of Gaussian distribution)
Trains faster somehow, might be due to initialization strategy(?)



GLU (Gated Linear Unit)

- A gated dense layer

GLU(x) = (Wx+b)*sigmoid(Vx+c)



https://jcarlosroldan.com/post/348/what-is-swiglu

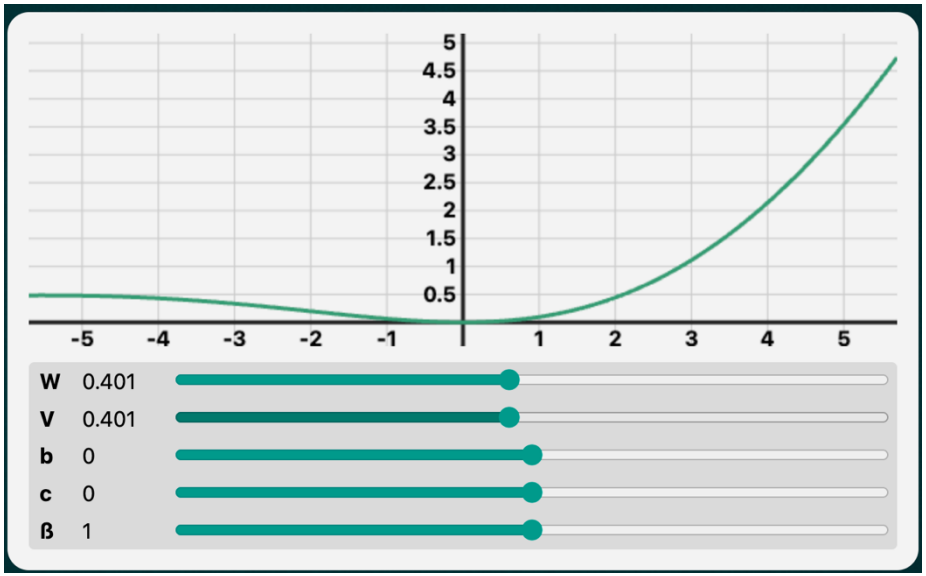
SwiGLU

- GLU with a Swish/SiLU gating function
- Many LLMs use this. Example: Llama, Qwen3, etc.

GLU(x) = (Wx+b)*sigmoid(Vx+c)

Swish(x) = x*sigmoid(Bx)

SwiGLU(x) = (Wx+b)*Swish(Vx+c)



https://jcarlosroldan.com/post/348/what-is-swiglu

Notes on non-linearity

- Start with ReLU
  - No parameter to tune, many papers/settings uses this so it might work better with the matched settings
  - Try LeakyReLU or GELU afterwards

Initialization

- The starting point of your descent
- Important due to local minimas
- Not as important with large networks AND big data
- Now usually initialized randomly
  - One strategy (Xavier init)

    $$var(w) = 2/(fan_in + fan_out)$$
  - For ReLUs (He init)

    $$var(w) = 2/(fan_in)$$
  - Or use a pre-trained network as initialization

X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. 2010
https://proceedings.mlr.press/v9/glorot10a.html
Kaiming He, Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. 2015
https://arxiv.org/abs/1502.01852

Stochastic gradient descent (SGD)

- Consider you have one million training examples
  - Gradient descent computes the objective function of all samples, then decide direction of descent
    - Takes too long
  - SGD computes the objective function on subsets of samples
    - The subset should not be biased and properly randomized to ensure no correlation between samples
- The subset is called a mini-batch
- Size of the mini-batch determines the training speed and accuracy
  - Usually somewhere between 32-1024 samples per mini-batch
    - Generally, works well with default settings
- Definition: 1 batch vs 1 epoch

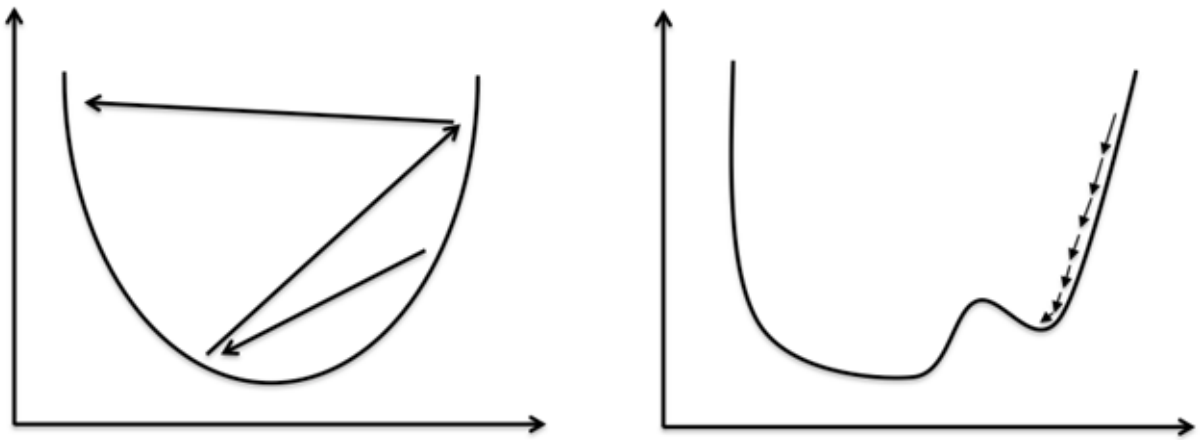
Self regularizing property of SGD

- SGD by its randomized nature does not overfit (as fast)
  - Considered as an implicit regularization (no change in the loss)

https://cbmm.mit.edu/sites/default/files/publications/CBMM-Memo-067-v3.pdf

Learning rate

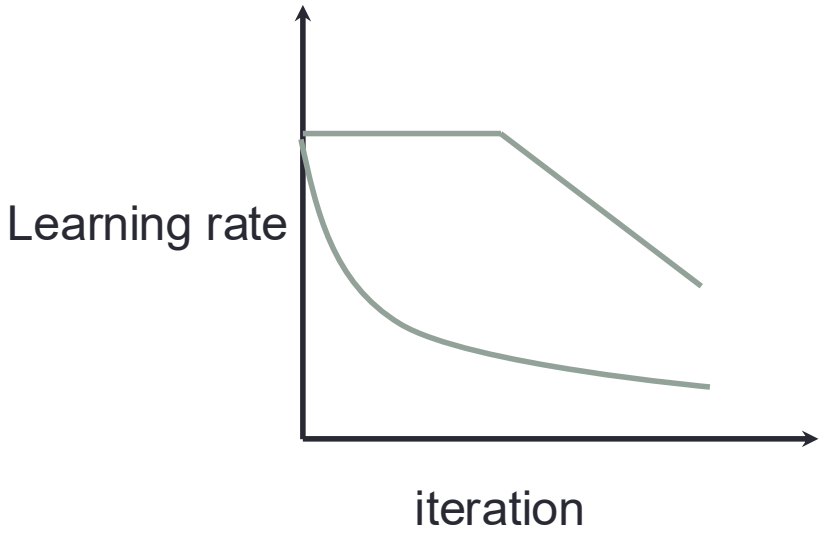
- How fast to go along the gradient direction is controlled by the learning rate
- Too large models diverge
- Too small the model get stuck in local minimas and takes too long to train



http://sebastianraschka.com/Articles/2015_singlelayer_neurons.html

Learning rate scheduling

- Usually starts with a large learning rate then gets smaller later
- Depends on your task
- Automatic ways to adjust the learning rate : Adagrad, Adam, etc. (still need scheduling still)



Learning rate strategies (annealing)

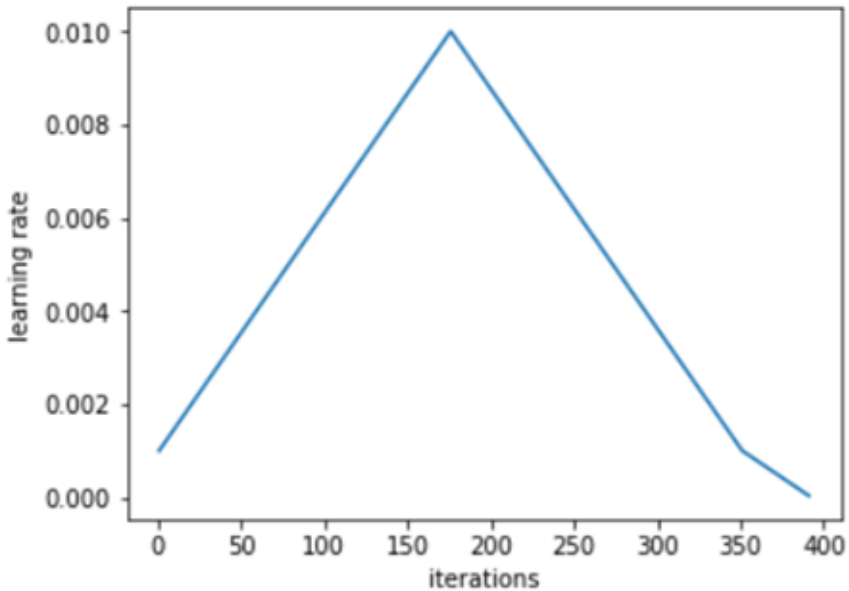
- Step decay: reduce learning rate by x after y epochs
- New bob method: half learning rate every time the validation error goes up. Only plausible in larger tasks
- Exponential decay: multiplies the learning rate by exp(-rate*epoch number)

Learning rate warm up

Initial point of the network can be at a bad spot.

Try not to go to fast - has a warmup period.

Useful for large datasets, or adaption (transfer learning)



Potentially leads to easier convergence and better accuracy – see paper below on why

See links below for methods to select the shape of the triangle

https://sgugger.github.io/the-1cycle-policy.html#the-1cycle-policy

Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour

Cyclical Learning Rates for Training Neural Networks

https://arxiv.org/abs/2406.09405

Notes on SGD

- SGD minimizes the loss function using just the first order derivative that is

$$L(\theta + \alpha\Delta\theta) = L(\theta) + \alpha L'(\theta)\Delta\theta + \frac{1}{2!}\alpha^2 L''(\theta)(\Delta\theta)^2 + \ldots$$

is approximate as

$$L(\theta + \alpha\Delta\theta) \simeq L(\theta) + \alpha L'(\theta)\Delta\theta$$

$$\nabla^2 f(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial^2 f}{\partial\theta_1\partial\theta_1} & \cdots & \frac{\partial^2 f}{\partial\theta_1\partial\theta_n} \\ \vdots & & \vdots \\ \frac{\partial^2 f}{\partial\theta_n\partial\theta_1} & \cdots & \frac{\partial^2 f}{\partial\theta_n\partial\theta_n} \end{bmatrix}$$

because computing the second order gradient is very expensive.

Newer techniques use tricks to try to compensate for this approximation

Momentum

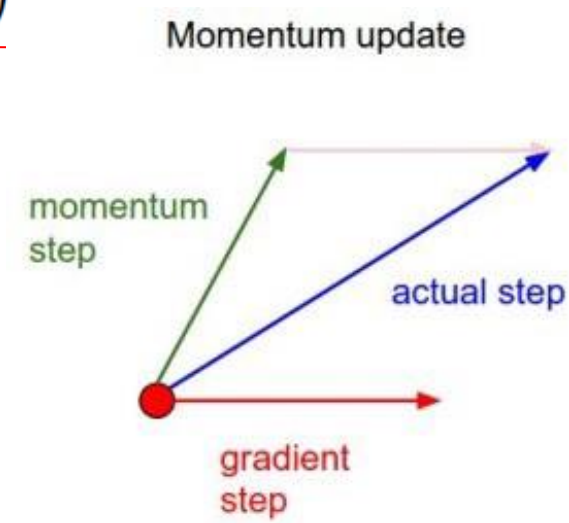
- Gradient descent can get stuck on small local minimas
  - Or slow down at saddle points
- Have concept of speed
- Can also consider as moving average of gradient estimate

speed Momentum rate gradient

$$V_t = \beta V_{t-1} + (1-\beta)\nabla_w L(W, X, y)$$

$$W = W - \alpha V_t$$

learning rate



http://cs231n.github.io/neural-networks-3/

Nesterov Momentum

Old

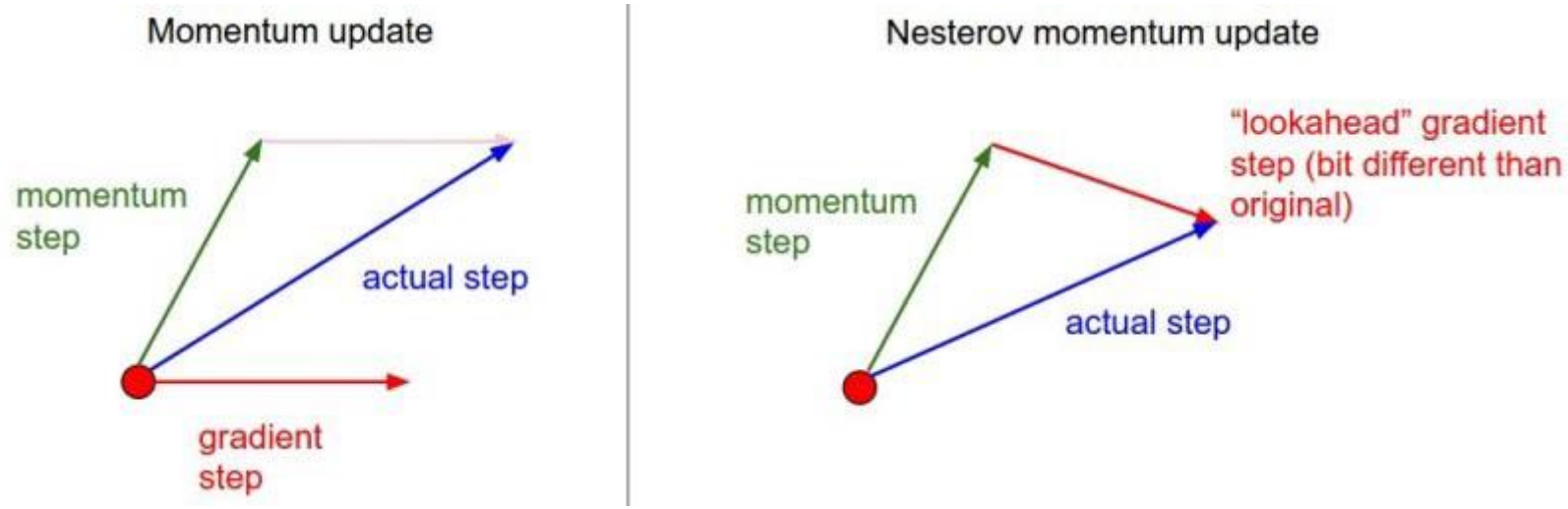
$$V_t = \beta V_{t-1} + (1-\beta)\nabla_w L(W, X, y)$$
$$W = W - \alpha V_t$$

W is about to be at W-βV due to momentum

- Momentum with look ahead.
  - Compute gradient as if we took an additional step

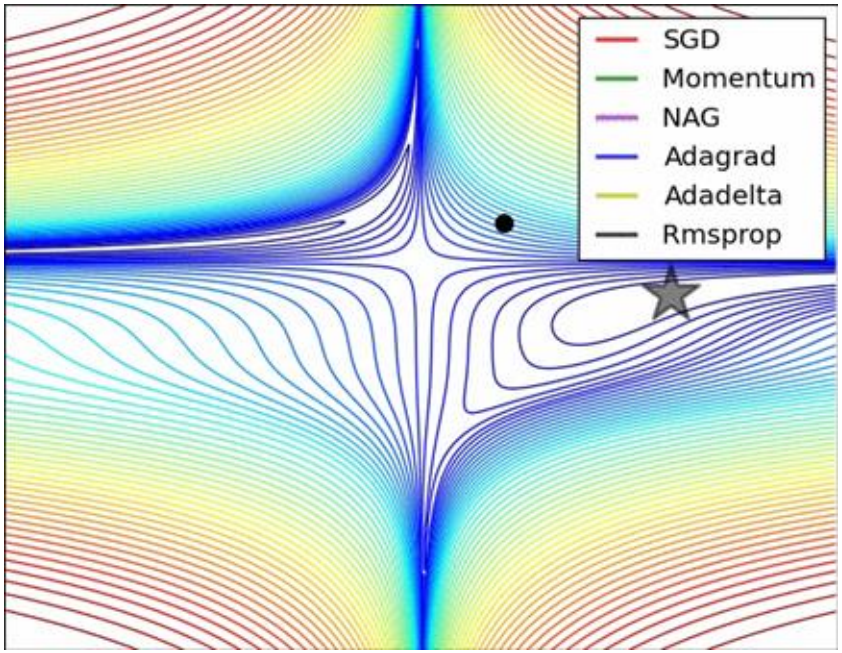
$$V_t = \beta V_{t-1} + \alpha \nabla_w L(\underline{W - \beta V_{t-1}}, X, y)$$
$$W = W - V_t$$

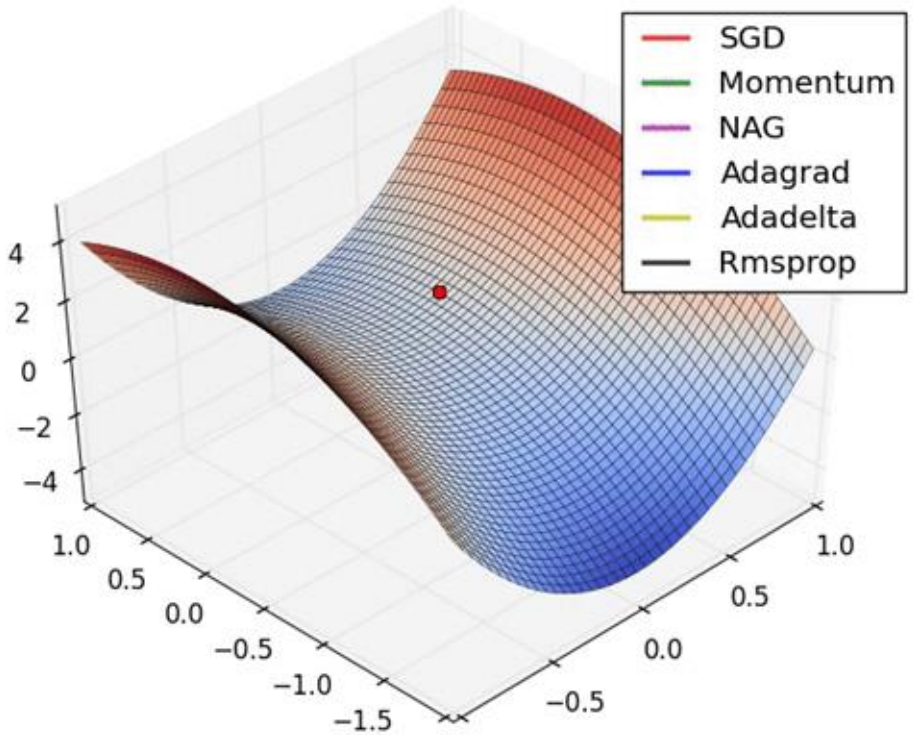
gradient is computed as if we took a step



http://cs231n.github.io/neural-networks-3/

Optimization method and speed





http://ruder.io/optimizing-gradient-descent/index.html#whichoptimizertochoose

Adaptive learning rates

How to have the updates be different for different layers?

LR of each weight is scaled by the size of the gradient

Can we trust the moment estimates?

Decay running estimates of the gradient size/momentums

RMSProp (normalized by past gradient size + moving average decay)

Adam (normalized by past gradient + decay both grad and momentum)

AdamW (Adam but deal with weight decay differently)

You still need to do learning rate scheduling

More details see

https://www.ruder.io/optimizing-gradient-descent/
https://www.ruder.io/deep-learning-optimization-2017/
https://towardsdatascience.com/why-adamw-matters-736223f31b5d

Image from https://stackoverflow.com/questions/64621585/adamw-and-adam-with-weight-decay

Adam vs AdamW

Adam

for $t = 1$ **to** $\ldots$ **do**

  if *maximize* :

    $g_t \leftarrow -\nabla_\theta f_t(\theta_{t-1})$

  else

    $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$

  if $\lambda \neq 0$

    $g_t \leftarrow g_t + \lambda\theta_{t-1}$

  $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t$

  $v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2$

  $\widehat{m_t} \leftarrow m_t / (1-\beta_1^t)$

  $\widehat{v_t} \leftarrow v_t / (1-\beta_2^t)$

  if *amsgrad*

    $\widehat{v_t}^{max} \leftarrow \max(\widehat{v_t}^{max}, \widehat{v_t})$

    $\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m_t} / (\sqrt{\widehat{v_t}^{max}} + \epsilon)$

  else

    $\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m_t} / (\sqrt{\widehat{v_t}} + \epsilon)$

1. g is the gradient

2. g is augmented with weight decay term

3. m (momentum) is the moving average of g

4. v (variance) is the moving average of the gradient size

5. v and m are biased correct because moving average will overemphasize the starting time steps

Amsgrad is a tweak from Adam

AdamW

for $t = 1$ **to** $\ldots$ **do**

  if *maximize* :

    $g_t \leftarrow -\nabla_\theta f_t(\theta_{t-1})$

  else

    $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$

  $\theta_t \leftarrow \theta_{t-1} - \gamma\lambda\theta_{t-1}$

  $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t$

  $v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2$

  $\widehat{m_t} \leftarrow m_t / (1-\beta_1^t)$

  $\widehat{v_t} \leftarrow v_t / (1-\beta_2^t)$

  if *amsgrad*

    $\widehat{v_t}^{max} \leftarrow \max(\widehat{v_t}^{max}, \widehat{v_t})$

    $\theta_t \leftarrow \theta_t - \gamma \widehat{m_t} / (\sqrt{\widehat{v_t}^{max}} + \epsilon)$

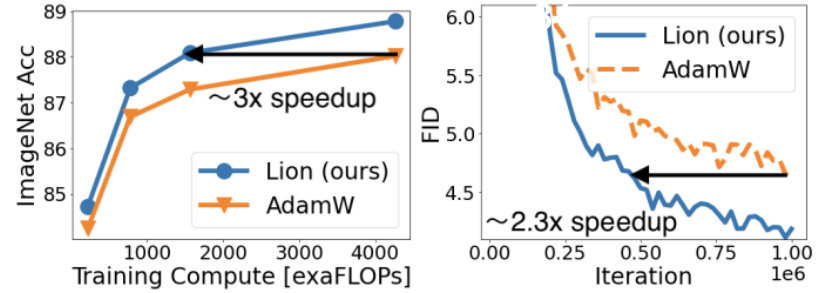
  else

    $\theta_t \leftarrow \theta_t - \gamma \widehat{m_t} / (\sqrt{\widehat{v_t}} + \epsilon)$

In AdamW weight decay term is used to change the weights directly, so it does not effect the momentum

6. The learning rate is normalized by the size of the gradient

Lion optimizer



- Optimizer found via Evolutionary algorithms.
- Keep track of only the momentum. Update using only the direction of the movement (sign). Step size mostly determined by learning rate.
- Reduce memory requirements (no need to keep track of the square) and usually offer better generalization.
- Has a version that decoupled weight decay.

Algorithm 1 AdamW Optimizer

given $\beta_1, \beta_2, \epsilon, \lambda, \eta, f$
initialize θ_0, $m_0 \leftarrow 0$, $v_0 \leftarrow 0$, $t \leftarrow 0$
while θ_t not converged **do**
  $t \leftarrow t + 1$
  $g_t \leftarrow \nabla_\theta f(\theta_{t-1})$
  update EMA of g_t **and** g_t^2
  $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$
  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
  bias correction
  $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$
  $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$
  update model parameters
  $\theta_t \leftarrow \theta_{t-1} - \eta_t (\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda \theta_{t-1})$
end while
return θ_t

Algorithm 2 Lion Optimizer (ours)

given $\beta_1, \beta_2, \lambda, \eta, f$
initialize θ_0, $m_0 \leftarrow 0$
while θ_t not converged **do**
  $g_t \leftarrow \nabla_\theta f(\theta_{t-1})$
  update model parameters
  $c_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$
  $\theta_t \leftarrow \theta_{t-1} - \eta_t (\text{sign}(c_t) + \lambda \theta_{t-1})$
  update EMA of g_t
  $m_t \leftarrow \beta_2 m_{t-1} + (1 - \beta_2) g_t$
end while
return θ_t

https://arxiv.org/abs/2302.06675

Learning rate tricks

- At least decay the learning rate
  - Monitor validation set performance
- If the loss never goes down -> decrease the learning rate (by factor of 10)
- Start with ADAM or ADAMW. DecoupledLionW if you want… (There is no guarantee which one will be better)
- Learning rate is the most important hyperparameter that will affect your model performance.

Learning rate and batch size

- Learning rate and batch size interacts with each other
  - Larger batch has low variance -> can use larger learning rate
    - Increase batch size by k, increase learning rate by k (can also scale the momentum etc, see papers)
      - https://arxiv.org/abs/1706.02677
      - https://arxiv.org/abs/1711.00489

- Set batch size first
  - largest to fit the GPU (trains faster, and get better accuracy)
- Then set your learning rate
  - Still need to tune. Above theory is just a guideline

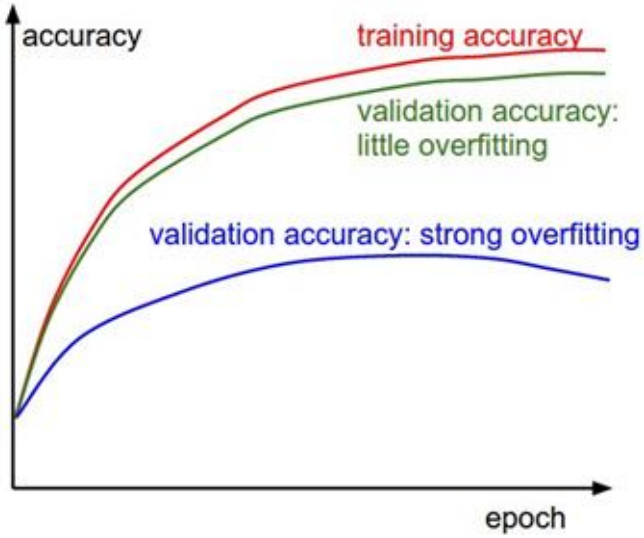
Small batchsize training

- Gradient accumulation
  - Accumulate gradients over multiple small batches before applying an parameter update
- Retune all learning rate hyperparamters

How to Train BERT with an Academic Budget https://arxiv.org/pdf/2104.07705.pdf

Overfitting

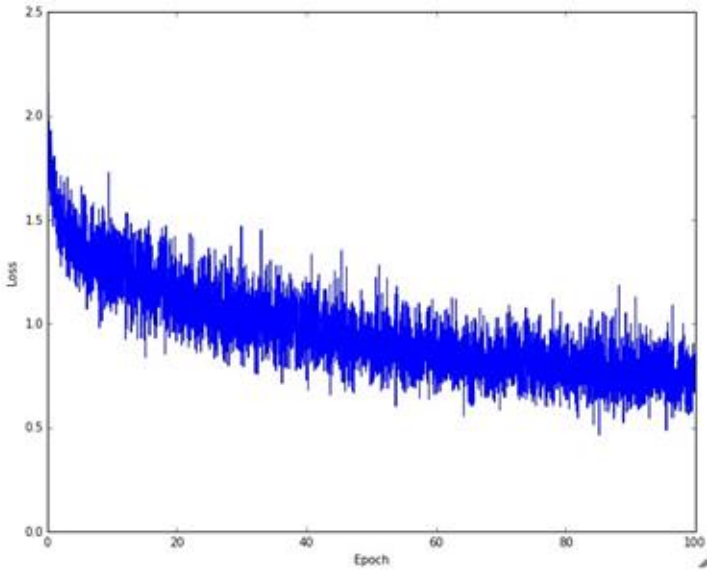
- You can keep doing back propagation forever!
- The training loss will always go down
- But it overfits
- Need to monitor performance on a held-out set
- Stop or decrease learning rate when overfit happens



http://cs231n.github.io/neural-networks-3/

Monitoring performance

- Monitor performance on a dev/validation set
  - This is NOT the test set
- Can monitor many criterions
  - Loss function
  - Classification accuracy
- Sometimes these disagree
- Actual performance can be noisy, need to see the trend



http://cs231n.github.io/neural-networks-3/

Dropout

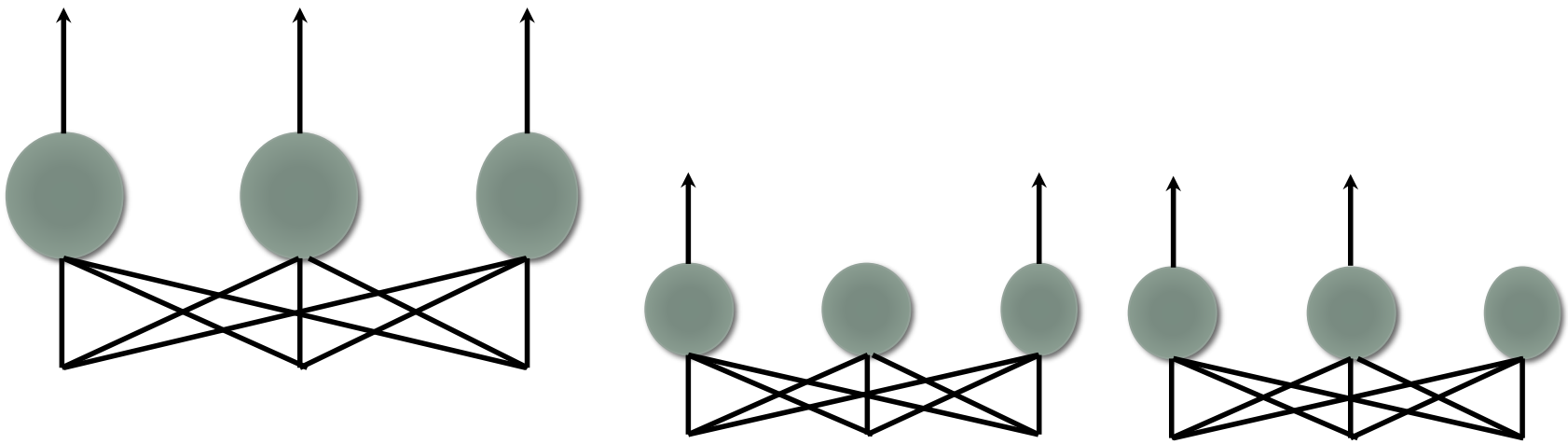
An implicit regularization technique for reducing overfitting

Randomly turn off different subset of neurons during training

Network no longer depend on any particular neuron

Force the model to have redundancy – robust to any corruption in input data

A form of performing model averaging (ensemble of experts)



Model

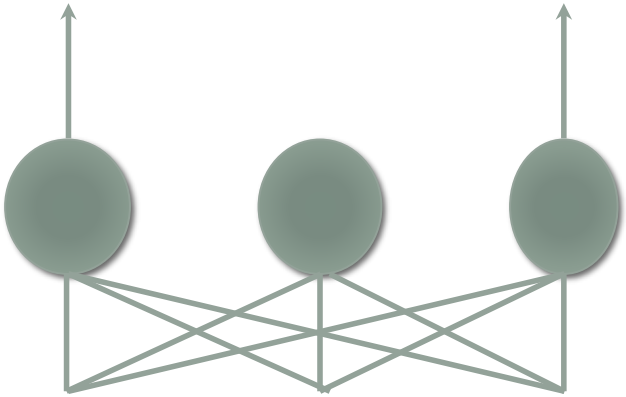
Dropout rate of 0.33

Hinton, Geoffrey "Improving neural networks by preventing co-adaptation of feature detectors" 2012

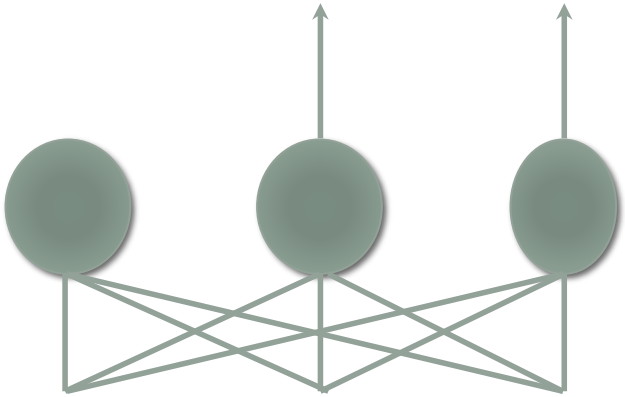
Dropout training time

Drop out rate 0.33

Randomly removed outputs
for each training sample



x1

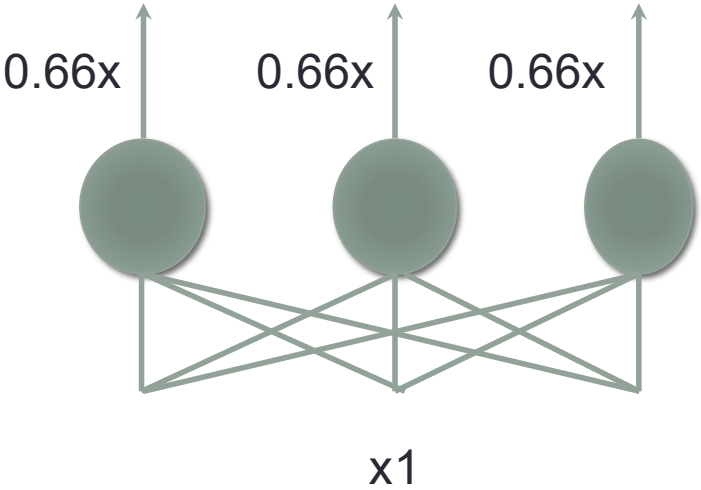


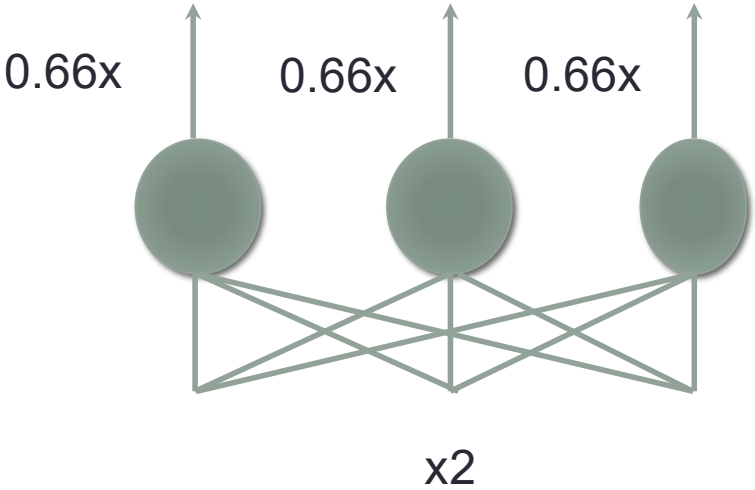
x2

Dropout test time

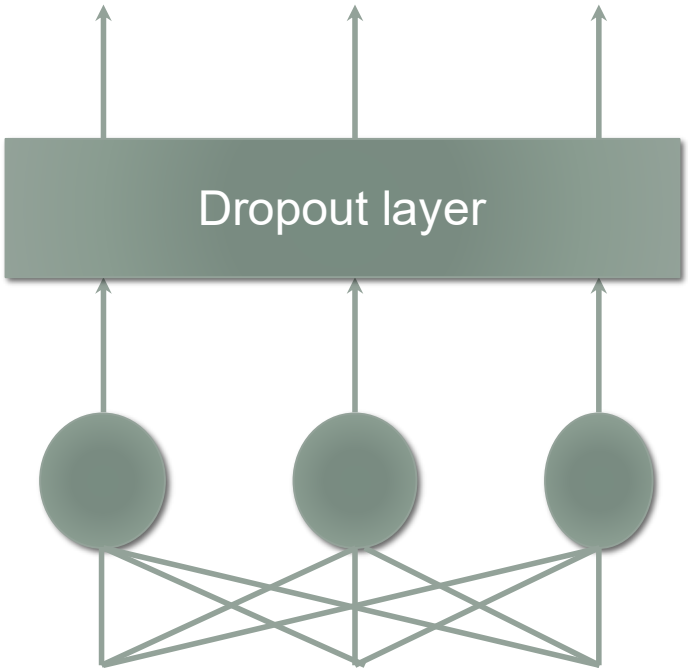
Dropout rate 0.33

Scale outputs so the output contribution is around the same





Dropout implementation



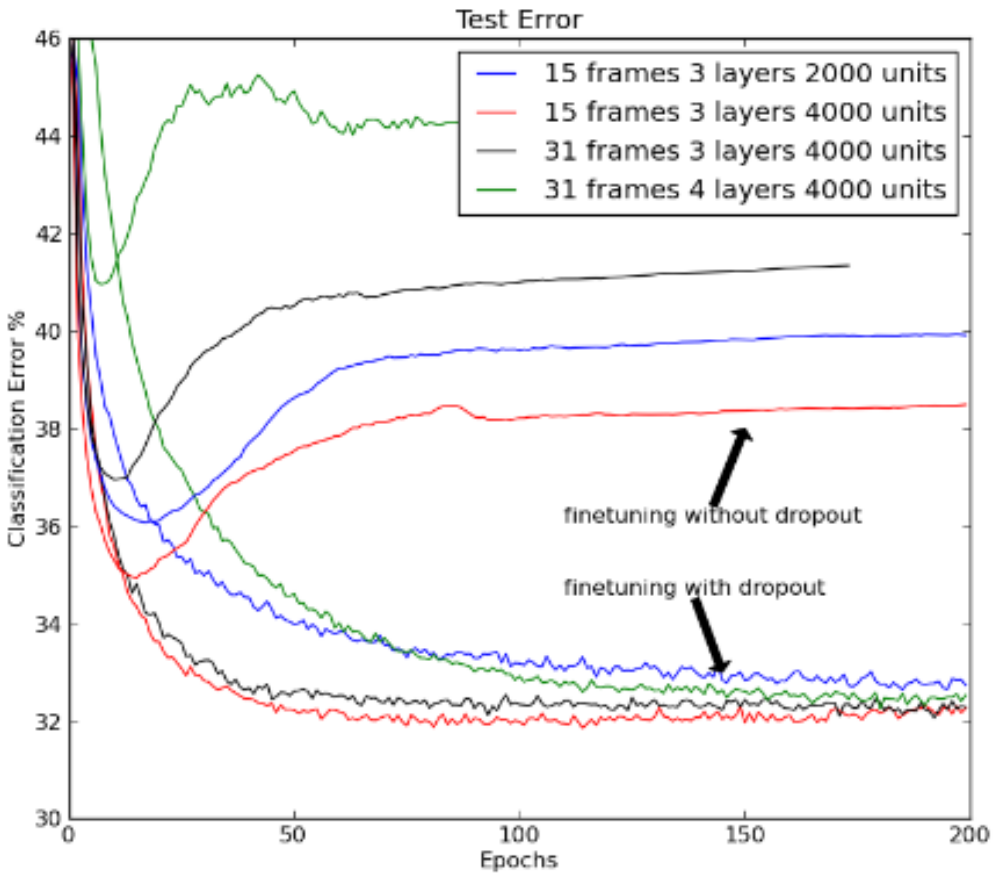
x1

<u>Dropout layer</u>
Just another layer that drops inputs

<u>Inverted dropout</u>
A variant of dropout that scales the dropout at training time, so that you don't have to scale at test time.

Dropout on TIMIT

- A phoneme recognition task



Hinton, Geoffrey "Improving neural networks by preventing co-adaptation of feature detectors" 2012

Batch normalization

- Recent technique for (implicit) regularization
- Normalize every mini-batch at various batch norm layers to standard Gaussian (different from global normalization of the inputs)
- Place batch norm layers before non-linearities
- Faster training and better generalizations

For each mini-batch that goes through batch norm

1. Normalize by the mean and variance of the mini-batch for each dimension
2. Shift and scale by learnable parameters

$$\hat{x} = \frac{x - \mu_b}{\sigma_b}$$

$$y = \alpha \hat{x} + \beta$$

Replaces dropout in some networks

https://arxiv.org/abs/1502.03167

Tips to tune

- Feeling and experience ¯\(°_o)/¯
- Take numbers from papers
- Grid search
  - Heuristic search
  - Random search
  - Genetic Algorithm
- Picking the right type of model is more important than picking the right number of neurons
  - Inductive bias
- Tips for debugging (will make more sense next class)
  - http://karpathy.github.io/2019/04/25/recipe/

Summary

- Layers
  - Dense/fully connected
  - Softmax
  - Nonlinearity
  - Dropout
  - Batchnorm
- Loss
  - training loss vs validation loss
  - Cross entropy vs MSE
- Optimizers
  - Backprop (autograd)
  - SGD, Adam, AdamW, Lion
  - Weight decay
- Learning rate scheduling